APPLICATION EXAMPLE

WinCC OA GraphQL & REST API Server

SIMATIC WinCC Open Architecture

Legal information

Use of application examples

Application examples illustrate the solution of automation tasks through an interaction of several components in the form of text, graphics, and/or software modules. The application examples are a free service by Siemens AG and/or a subsidiary of Siemens AG ("Siemens"). They are non-binding and make no claim to completeness or functionality regarding configuration and equipment. The application examples merely offer help with typical tasks; they do not constitute customer-specific solutions. You yourself are responsible for the proper and safe operation of the products in accordance with applicable regulations and must also check the function of the respective application example and customize it for your system.
Siemens grants you the non-exclusive, non-sublicensable and non-transferable right to have the application examples used by technically trained personnel. Any change to the application examples is your responsibility. Sharing the application examples with third parties or copying the application examples or excerpts thereof is permitted only in combination with your own products. The application examples are not required to undergo the customary tests and quality inspections of a chargeable product; they may have functional and performance defects as well as errors. It is your responsibility to use them in such a manner that any malfunctions that may occur do not result in property damage or injury to persons.

Disclaimer of liability

Siemens shall not assume any liability, for any legal reason whatsoever, including, without limitation, liability for the usability, availability, completeness, and freedom from defects of the application examples as well as for related information, configuration and performance data and any damage caused thereby. This shall not apply in cases of mandatory liability, for example under the German Product Liability Act, or in cases of intent, gross negligence, or culpable loss of life, bodily injury or damage to health, non-compliance with a guarantee, fraudulent non-disclosure of a defect, or culpable breach of material contractual obligations. Claims for damages arising from a breach of material contractual obligations shall however be limited to the foreseeable damage typical of the type of agreement, unless liability arises from intent or gross negligence or is based on loss of life, bodily injury, or damage to health. The foregoing provisions do not imply any change in the burden of proof to your detriment. You shall indemnify Siemens against existing or future claims of third parties in this connection except where Siemens is mandatorily liable.
By using the application examples you acknowledge that Siemens cannot be held liable for any damage beyond the liability provisions described.

Other information

Siemens reserves the right to make changes to the application examples at any time without notice. In case of discrepancies between the suggestions in the application examples and other Siemens publications such as catalogs, the content of the other documentation shall have precedence.
The Siemens terms of use (https://support.industry.siemens.com) shall also apply.

Security information

Siemens provides products and solutions with industrial cybersecurity functions that support the secure operation of plants, systems, machines and networks.

In order to protect plants, systems, machines and networks against cyber threats, it is necessary to implement – and continuously maintain – a holistic, state-of-the-art industrial cybersecurity concept. Siemens' products and solutions constitute one element of such a concept.

Customers are responsible for preventing unauthorized access to their plants, systems, machines and networks. Such systems, machines and components should only be connected to an enterprise network or the internet if and to the extent such a connection is necessary and only when appropriate security measures (e.g. firewalls and/or network segmentation) are in place.

For additional information on industrial cybersecurity measures that may be implemented, please visit www.siemens.com/cybersecurity-industry.

Siemens' products and solutions undergo continuous development to make them more secure. Siemens strongly recommends that product updates are applied as soon as they are available and that the latest product versions are

used. Use of product versions that are no longer supported, and failure to apply the latest updates may increase customer's exposure to cyber threats.

To stay informed about product updates, subscribe to the Siemens Industrial Cybersecurity RSS Feed under https://www.siemens.com/cert.

Third-Party Software Information

Note to Resellers: Please pass on this document to your customer to avoid license infringements.

This product, solution or service ("Product") contains third-party software components listed in this document. These components are Open Source Software licensed under a license approved by the Open Source Initiative (www.opensource.org) or similar licenses as determined by SIEMENS ("OSS") and/or commercial or freeware software components. With respect to the OSS components, the applicable OSS license conditions prevail over any other terms and conditions covering the Product. The OSS portions of this Product are provided royalty-free and can be used at no charge.

If SIEMENS has combined or linked certain components of the Product with/to OSS components licensed under the GNU LGPL version 2 or later as per the definition of the applicable license, and if use of the corresponding object file is not unrestricted ("LGPL Licensed Module", whereas the LGPL Licensed Module and the components that the LGPL Licensed Module is combined with or linked to is the "Combined Product"), the following additional rights apply, if the relevant LGPL license criteria are met: (i) you are entitled to modify the Combined Product for your own use, including but not limited to the right to modify the Combined Product to relink modified versions of the LGPL Licensed Module, and (ii) you may reverse-engineer the Combined Product, but only to debug your modifications. The modification right does not include the right to distribute such modifications and you shall maintain in confidence any information resulting from such reverse-engineering of a Combined Product.

Certain OSS licenses require SIEMENS to make source code available, for example, the GNU General Public License, the GNU Lesser General Public License and the Mozilla Public License. If such licenses are applicable and this Product is not shipped with the required source code, a copy of this source code can be obtained by anyone in receipt of this information during the period required by the applicable OSS licenses by contacting the following address:

Siemens AG
LC TEC IT&SL
Werner-von-Siemens Str. 60
91052 Erlangen
Germany

Keyword: Open Source Request (please specify Product name and version, if applicable)

SIEMENS may charge a handling fee of up to 5 EUR to fulfil the request.

Warranty regarding further use of the Open Source Software

SIEMENS' warranty obligations are set forth in your agreement with SIEMENS. SIEMENS does not provide any warranty or technical support for this Product or any OSS components contained in it if they are modified or used in any manner not specified by SIEMENS. The license conditions listed below may contain disclaimers that apply between you and the respective licensor. For the avoidance of doubt, SIEMENS does not make any warranty commitment on behalf of or binding upon any third party licensor.

Open Source Software and/or other third-party software contained in this Product:

Please note the following license conditions and copyright notices applicable to Open Source Software and/or other components (or parts thereof):

Component	Open Source Software [Yes/No]	Acknowledgements/Comment	License conditions and copyright notices
@apollo/server 5.5.1	Yes	MIT	https://github.com/apollographql/apollo-server/blob/main/LICENSE
@as-integrations/express5 1.1.2	Yes	MIT	https://github.com/apollo-server-integrations/apollo-server-integration-express5/blob/main/LICENSE

@graphql-tools/schema 10.0.33	Yes	MIT	https://github.com/ardatan/graphql-tools/blob/master/LICENSE
cors 2.8.6	Yes	MIT	https://github.com/expressjs/cors/blob/master/LICENSE
dotenv 17.4.2	Yes	BSD-2-Clause	https://github.com/motdotla/dotenv/blob/master/LICENSE
express 5.2.1	Yes	MIT	https://github.com/expressjs/express/blob/master/LICENSE
graphql 16.14.1	Yes	MIT	https://github.com/graphql/graphql-js/blob/main/LICENSE
graphql-ws 6.0.8	Yes	MIT	https://github.com/enisdenjo/graphql-ws/blob/master/LICENSE.md
js-yaml 4.2.0	Yes	MIT	https://github.com/nodeca/js-yaml/blob/master/LICENSE
swagger-ui-express 5.0.1	Yes	MIT	https://github.com/scottie1984/swagger-ui-express/blob/master/LICENSE
uuid 14.0.0	Yes	MIT	https://github.com/uuidjs/uuid/blob/main/LICENSE.md
ws 8.21.0	Yes	MIT	https://github.com/websockets/ws/blob/master/LICENSE
jsonwebtoken 9.0.3	Yes	MIT	https://github.com/auth0/node-jsonwebtoken/blob/master/LICENSE

Table 0-1 Third-Party Software Overview

Contents

1. Introduction

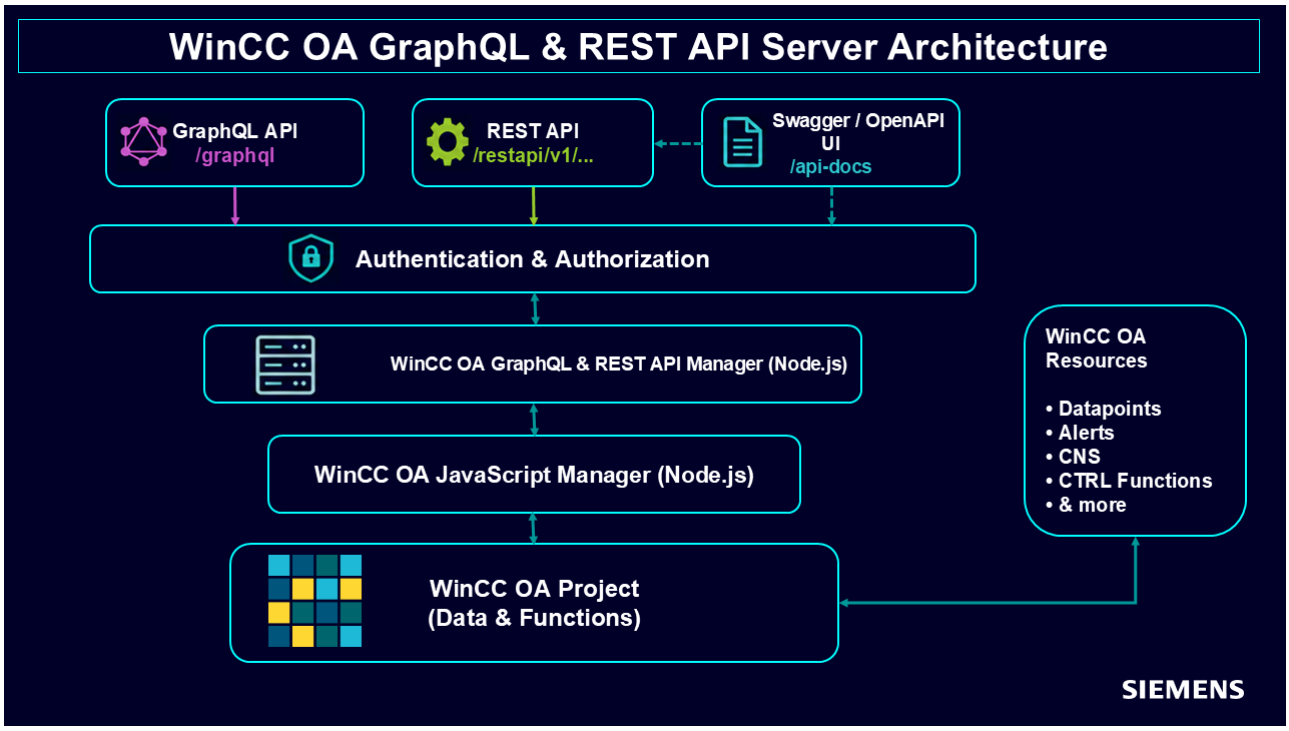
1.1. Overview

This application example demonstrates how a SIMATIC WinCC Open Architecture (WinCC OA) system can be extended with a modern API layer using a Node.js-based JavaScript Manager.

The provided solution enables standardized and secure access to WinCC OA data and functionality via:

- A **GraphQL API** supporting flexible queries and real-time data subscriptions
- A **REST API** providing conventional HTTP-based access to system resources
- **JWT-based authentication** with role-based access control (e.g., administrator and read-only users)
- An **interactive API documentation interface (Swagger UI)** for testing and exploration

By introducing this API layer, external systems such as web applications, dashboards, or third-party services can interact with WinCC OA in a controlled and secure manner.



1.2. Components used

1.2.1. WinCC OA

This application example has been created with the following hardware and software components:

Component	Number	Article number
WinCC OA 3.21, Server Basis	1	6AV6355-1AA50-1BA0
WinCC OA V3.21, Para Standard	1	6AV6355-1AA50-1CH0

Table 1-1 WinCC OA licenses

You can purchase these components from the Siemens Industry Mall.

This application example consists of the following components:

Component	File name	Note
Manual	`WinCCOAGraphqlRestApiExample.pdf`	This document
Subproject WinCC OA	`WinCCOAGraphqlRestApi.zip`	Subproject for WinCC OA

Table 1-2 Application Example components

1.2.2. API Server (Node.js Environment)

The API server is implemented as a Node.js application and is executed within the WinCC OA JavaScript Manager.
It provides both REST and GraphQL interfaces for accessing WinCC OA data and functionality.
The implementation is based on the following technologies:

- Node.js runtime (provided by the WinCC OA JavaScript Manager)
- Apollo Server (used for implementing the GraphQL server interface)
- Express.js (used for implementing HTTP-based REST endpoints)
- GraphQL (used for flexible data queries and real-time subscriptions)
- graphql-ws (used for GraphQL WebSocket-based subscriptions)
- dotenv (used for environment-based configuration management)

The GraphQL interface additionally supports real-time subscriptions via WebSocket communication.

1.2.3. Client / Access Interfaces

The API can be accessed and tested using the following interfaces:

- **Swagger UI** (interactive API documentation accessible via a web browser, allowing exploration and execution of REST endpoints)
- **GraphQL endpoint** (accessible via a web browser or external GraphQL clients for executing queries and subscriptions)
- **REST API clients** (standard HTTP-based access using web browsers or command-line tools)

2. Engineering

2.1. System requirements

WinCC OA Version 3.21 P0 or higher installed and valid license (incl. Para)

- Installed WinCC OA JavaScript Manager for Node.js (see WinCC OA documentation: WinCC OA JavaScript Manager for Node.js)
- Minimal system requirements for WinCC OA (see WinCC OA Documentation)

2.2. Project integration

2.2.1. Installation WinCC OA

To carry out the installation of WinCC OA, please follow the steps in the WinCC OA documentation.
You can find the documentation under following link: Installation

Note:
Node.js is automatically installed with WinCC OA. No additional installation steps are required (see WinCC OA documentation: Node.js Installation).

On Linux systems, if a version of Node.js is already installed, it may not be updated automatically during the WinCC OA installation. In this case, ensure that Node.js is either updated to the latest version or uninstalled before installing WinCC OA with a JavaScript environment.

2.2.2. Subproject Integration

Unpack the supplied ZIP archive `WinCCOAGraphQLRestApisExample.zip` and register it as a subproject (see WinCC OA documentation: Register project.).
The ZIP-archive contains one main folder:

- `javascript` - JavaScript source (`index.js`) and support files for the Teams Integration Manager.

  Within the `javascript` directory, the following application folder is included:
  - `winccoa-graphql-server-main` – contains the Node.js-based API server implementation (including `index.js`) and all required supporting files.

2.2.3. Creating a WinCC OA Project

To create your own project and use it with the manager, steps form online documentation Create project can be followed. then include it in your project Change project properties (see “Include Subprojects” section).

2.3. Installation and Configuration

2.3.1. Installation of Dependencies

To prepare the WinCC OA GraphQL & REST API Server for operation, the required Node.js dependencies must be installed using npm.

Open a PowerShell terminal and navigate to the application directory:

```
cd "<subproject_path>\javascript\winccoa-graphql-server-main"
```

Before installation, the application directory contains the project source files and configuration templates only.

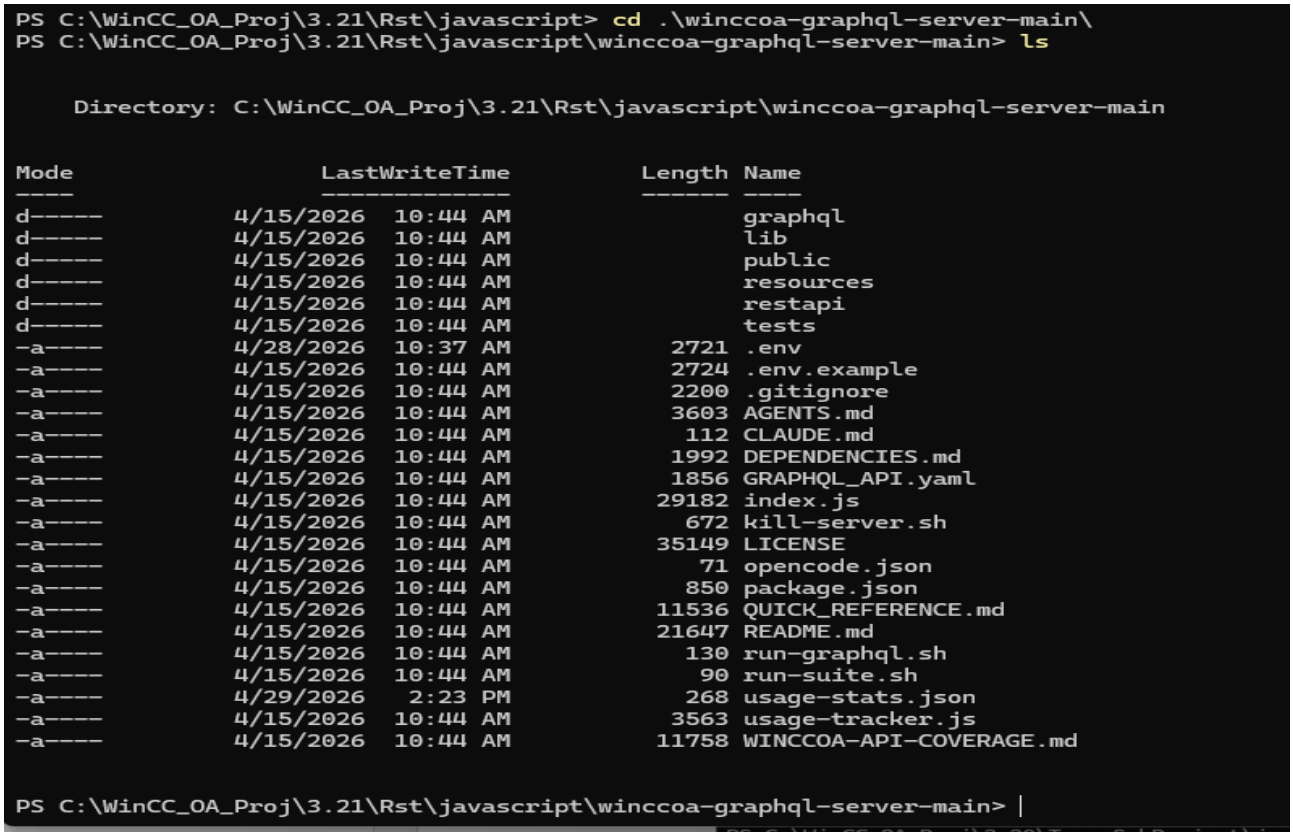


Figure 2-1 Application directory before executing npm install

- Run the “npm install” command to install all required packages:

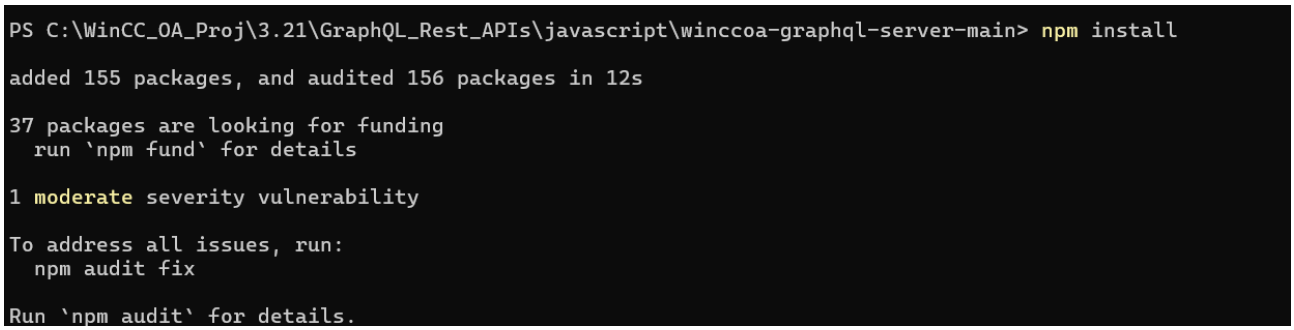


Figure 2-2 npm install completed

Running `npm install` reads the `package.json` file and installs all required dependencies for the API server. After successful installation:

- the `node_modules` directory is created,
- the `package-lock.json` file is generated or updated,
- and all required project dependencies become available.

The successful creation of the node_modules directory and the package-lock.json file can be verified directly in the application directory:
`<WinCC_OA_Project>\javascript\winccoa-graphql-server-main\`

2.3.2. Resolving Security Vulnerabilities

After installing the project dependencies, `npm` may report known package vulnerabilities during the installation process.

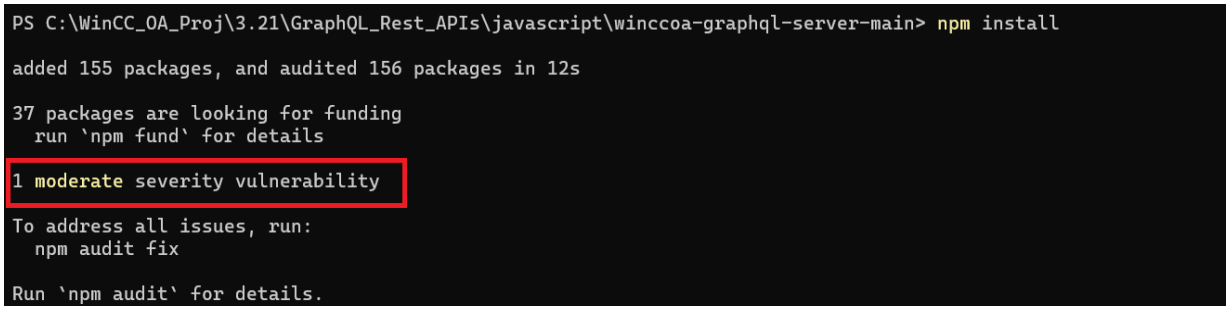


Figure 2-3 npm audit output showing detected package vulnerabilities

To automatically resolve compatible vulnerabilities, execute `npm audit fix`

This command updates compatible vulnerable dependencies and verifies the remaining vulnerability status after installation

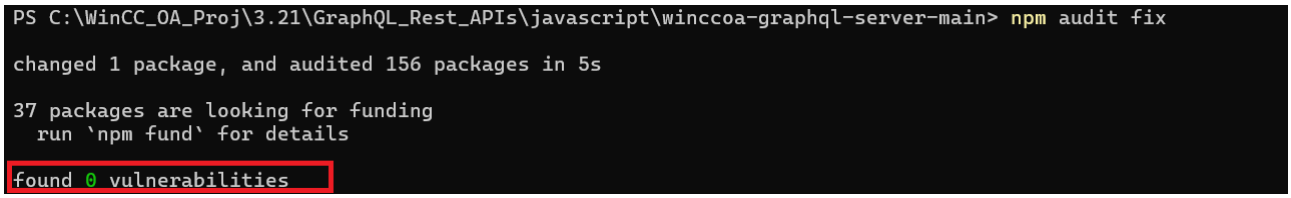


Figure 2-4 Successful npm audit fix execution showing no remaining vulnerabilities

A successful result is indicated by output similar to the following: found 0 vulnerabilities

2.3.3. Adapting the Startup Script for Windows

The startup configuration of the WinCC OA GraphQL & REST API Server is defined in the package.json file located in the application directory::

```
<WinCC_OA_Project>\javascript\winccoa-graphql-server-main\package.json
```

Open the package.json file and navigate to the scripts section.

The default configuration may contain a Linux-specific WinCC OA bootstrap path similar to the following:

```
start": "node \"/opt/WinCC_OA/3.20/javascript/winccoa-manager/lib/bootstrap.js\" -PROJ
\"TestMass\" -pmonIndex 8 index.js
```

Since this Linux-specific path does not exist on Windows systems the startup configuration must be adapted to the local WinCC OA installation directory and project configuration.

Example Windows configuration:

```
start": "node \"C:/Program Files/Siemens/WinCC_OA/3.21/javascript/winccoa-
manager/lib/bootstrap.js\" -PROJ \"GraphQL_Rest_APIs\" -pmonIndex 8 index.js
```

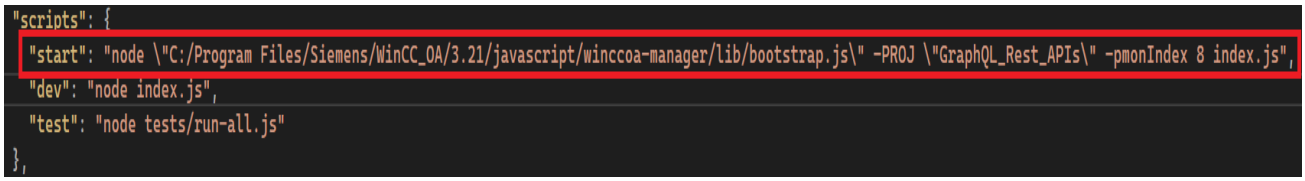


Figure 2-5 Adapting the Startup Script for the Operating System and Project Configuration

- WinCC OA bootstrap path – Path to the local WinCC OA JavaScript Manager bootstrap file
- -PROJ – Name of the WinCC OA project
- -pmonIndex – Unique and currently unused PMON manager index

2.3.4. Checking Whether the .env File Exists

The project requires the .env configuration file containing runtime and authentication settings.

Run the following command in the same folder:

Test-Path ".env"

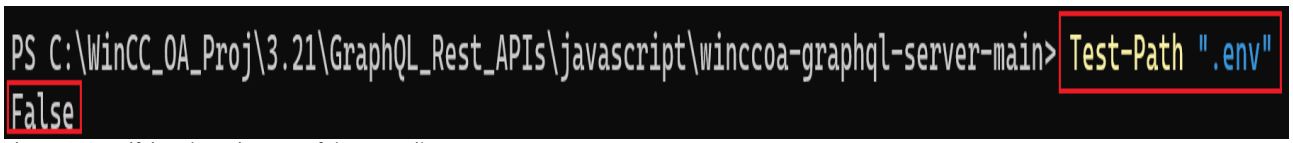


Figure 2-6 Verifying the Existence of the .env File

This means that the .env file does not exist yet.

The .env file is required because it contains important runtime configuration, such as:

- server port,
- JWT secret,
- admin credentials,
- read-only user credentials,
- and other environment-specific settings.

2.3.5. Creating the .env File

The GraphQL & REST API Server requires a local .env configuration file containing runtime and authentication settings.

Create the .env file from the provided .env.example template and verify that the file is available in the application directory.

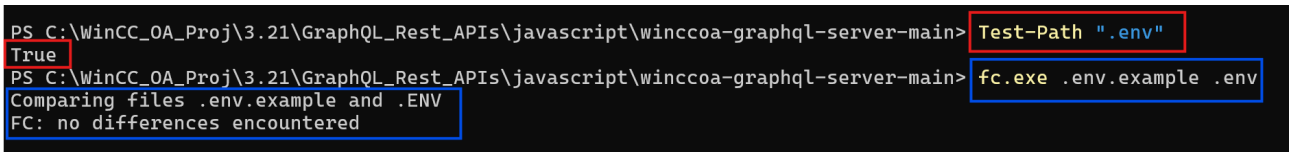


Figure 2-7 Creating and verifying the .env configuration file

```
# GraphQL Server Configuration
# Copy this file to .env and adjust the values

# Server Host
# Set to the interface to listen on. Default: 0.0.0.0 (all interfaces)
# Examples: 0.0.0.0 (all interfaces), localhost, 127.0.0.1, ::1 (IPv6)
GRAPHQL_HOST=0.0.0.0

# Server Port
GRAPHQL_PORT=4000

# Authentication Settings
# Set to true to disable all authentication (not recommended for production)
DISABLE_AUTH=false

# JWT Secret Key (change this in production!)
JWT_SECRET=your-secret-key-change-in-production

# Authentication Mode
# Controls how username/password logins are validated:
#   config  (default) — only env-var credentials (ADMIN_USERNAME / READONLY_USERNAME)
#   winccoa           — only WinCC OA user database (users defined in WinCC OA project)
#   both              — try env-var first, fall back to WinCC OA
AUTH_MODE=config

# Admin User Credentials (used when AUTH_MODE=config or AUTH_MODE=both)
# If set, these credentials can be used to login with full access
ADMIN_USERNAME=admin
ADMIN_PASSWORD=changeme

# Direct Access Token (non-JWT, not affected by AUTH_MODE)
# If set, this token can be used directly in the Authorization header for full access
# Example: Authorization: Bearer your-direct-access-token
DIRECT_ACCESS_TOKEN=

# Read-Only User Credentials (used when AUTH_MODE=config or AUTH_MODE=both)
# If set, these credentials provide read-only access (queries only, no mutations)
READONLY_USERNAME=readonly
READONLY_PASSWORD=readonly123

# Read-Only Direct Access Token (non-JWT, not affected by AUTH_MODE)
# If set, this token provides direct read-only access
# Example: Authorization: Bearer your-readonly-token
READONLY_TOKEN=

# WinCC OA Read-Only Users (used when AUTH_MODE=winccoa or AUTH_MODE=both)
# Comma-separated list of WinCC OA usernames that get 'readonly' role.
# All other WinCC OA users get 'admin' role.
# Example: WINCCOA_READONLY_USERS=guest,viewer
WINCCOA_READONLY_USERS=

# Token Expiry (in milliseconds)
# Default: 3600000 (1 hour)
TOKEN_EXPIRY_MS=3600000

# Logging Level
# Options: debug, info, warn, error
```

Figure 2-8 Example .env configuration template

At minimum, the following parameters should be verified or configured in the .env file:

`GRAPHQL_PORT`

`JWT_SECRET`

`ADMIN_USERNAME`

`ADMIN_PASSWORD`

`READONLY_USERNAME`

`READONLY_PASSWORD`

Parameter description:
`GRAPHQL_PORT` defines the API server port.
`JWT_SECRET` defines the secret used for JWT token generation.
`ADMIN_USERNAME` and `ADMIN_PASSWORD` define the administrative login credentials.
`READONLY_USERNAME` and `READONLY_PASSWORD` define the read-only login credentials.

2.3.6. API Server Configuration

Additional runtime behavior can be configured using optional .env parameters.

Example configuration:

```
LOG_LEVEL=info
CORS_ORIGIN=*
DISABLE_AUTH=false
TOKEN_EXPIRY_MS=3600000
```

Configuration overview:

- LOG_LEVEL controls the server logging level.
- CORS_ORIGIN defines the allowed cross-origin requests.
- DISABLE_AUTH enables or disables authentication.
- TOKEN_EXPIRY_MS defines the JWT token lifetime in milliseconds.

Note: Using "*" is only recommended for development or testing environments.

2.3.7. Authentication Configuration

The server supports:

- JWT-based authentication,
- username/password login,
- direct access tokens,
- read-only access,
- and WebSocket authentication for GraphQL subscriptions.

Optional token-based authentication parameters:

- DIRECT_ACCESS_TOKEN defines a predefined full-access token.
- READONLY_TOKEN defines a predefined read-only access token.

Read-only users are limited to query operations and cannot execute write operations. WebSocket-based GraphQL subscriptions are supported for read-only users when a valid authorization token is provided via the WebSocket connection parameters.

2.3.8. Security Recommendations

For productive deployments, the following recommendations should be considered:

- Replace the default JWT_SECRET with a strong project-specific secret.
- Avoid using DISABLE_AUTH=true outside development or testing environments.
- Restrict CORS_ORIGIN to trusted systems only.
- Use HTTPS when exposing the API externally.
- Protect API credentials and access tokens from unauthorized access.
- Do not use the default credentials that are in .env, replace them with your own credentials
- Rotate JWT secrets, access tokens, and passwords regularly according to project security policies.
- Place the API server behind a reverse proxy or firewall in productive environments.

2.4. API Server Startup and Verification

This chapter describes how to start the WinCC OA GraphQL & REST API Server and verify that the runtime environment was initialized successfully.

2.4.1. Starting the API Server

The WinCC OA GraphQL & REST API Server is started via a JavaScript Manager.

In the WinCC OA Console:

Add and start a new JavaScript Manager and configure the startup script in the "**Options**" field: `winccoa-graphql-server-main/index.js`.

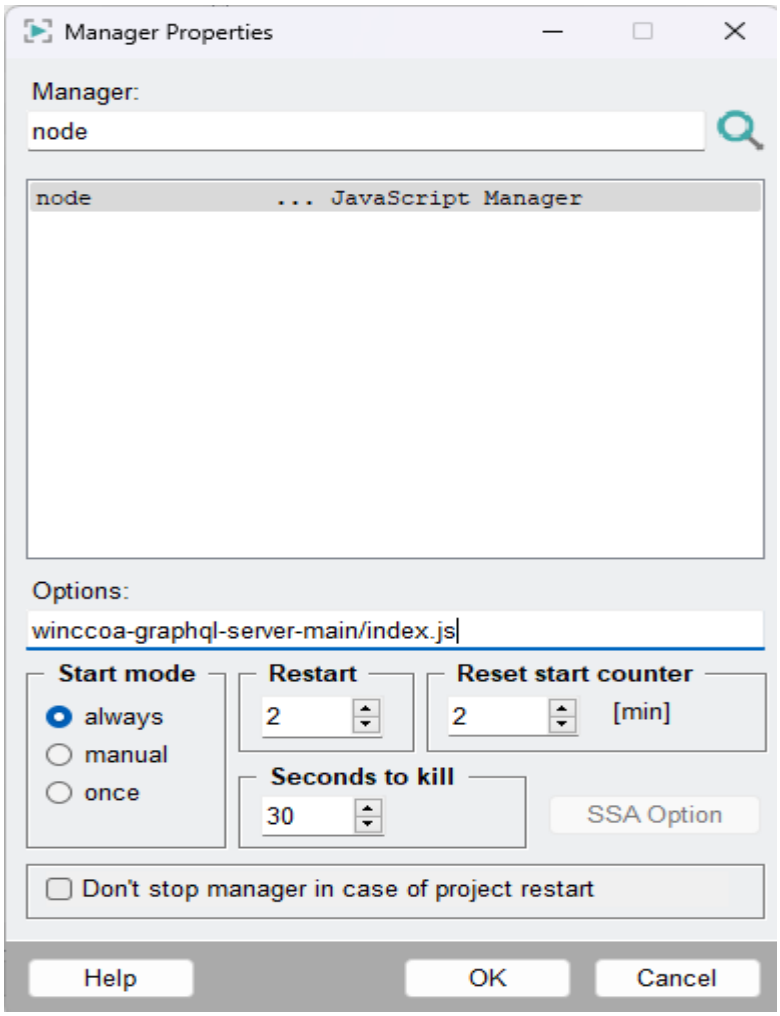


Figure 2-9 JavaScript Manager configured with the GraphQL & REST API Server startup script

After the manager has been added, start the JavaScript Manager from the WinCC OA Console.

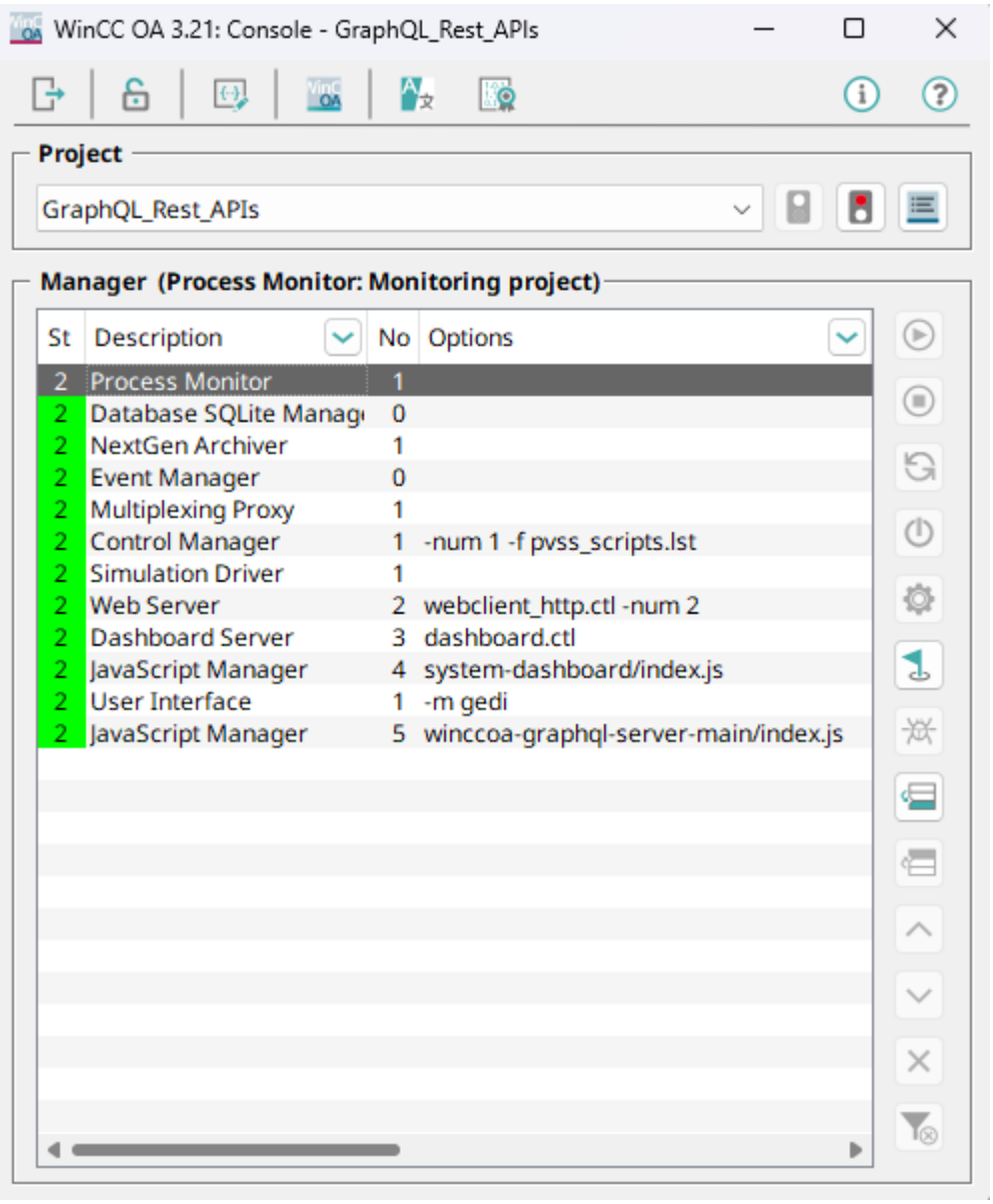


Figure 2-10 Running JavaScript Manager for the GraphQL & REST API Server

If the startup configuration is correct, the JavaScript Manager remains active and initializes the:

- GraphQL API,
- REST API,
- WebSocket subscription interface,
- and Swagger/OpenAPI documentation interface

2.4.2. Verifying Successful Startup

Successful startup can be verified in the WinCC OA Log Viewer.
Authentication is properly configured.
WinCC OA API Server Started

```
0/javascript, Looking for .env file at: C:\WinCC_OA_Proj\3.21\GraphQL_Rest_APIs\javascript\winccoa-graphq
0/javascript, ✅ .env file loaded successfully
0/javascript,    Loaded variables: GRAPHQL_HOST, GRAPHQL_PORT, DISABLE_AUTH, JWT_SECRET, AUTH_MODE, ADMIN
0/javascript, 📊 No existing usage statistics file found, starting fresh
0/javascript, Starting GraphQL server on 0.0.0.0:4000 with DISABLE_AUTH=false
0/javascript, 🌐 CORS Configuration: *
0/javascript, 🔐 Authentication Configuration:
0/javascript,    Admin Username: ✅ Set
0/javascript,    Admin Password: ✅ Set
0/javascript,    Direct Access Token: ❌ Not set
0/javascript,    Readonly Username: ✅ Set
0/javascript,    Readonly Token: ❌ Not set
0/javascript,    JWT Secret: ⚠️  Default (change in production)
0/javascript,    Token Expiry / Idle Timeout: 3600000ms (60 minutes)
0/javascript,    Auth Mode: config (config=env-var only, winccoa=WinCC OA users, both=env-var then WinCC
0/javascript,    Dev Login: ❌ Disabled
0/javascript, ✅ Authentication is properly configured.
0/javascript, 🏭 WinCC OA API Server Started
```

Figure 2-11 Successful startup of the GraphQL & REST API Server

These log entries confirm that:

- the .env configuration was loaded successfully,
- the authentication configuration was initialized correctly,
- and the API server started successfully.

2.4.3. Available API Endpoints

After successful startup, the following API endpoints are available:

Component	Description
http://localhost:4000/	Landing page
http://localhost:4000/graphql	GraphQL API endpoint
http://localhost:4000/restapi	REST API endpoint
http://localhost:4000/api-docs	Swagger/OpenAPI documentation
http://localhost:4000/openapi.json	OpenAPI specification
http://localhost:4000/restapi/health	REST API health check endpoint

3. Authentication and Authorization

This chapter describes the authentication and authorization mechanisms used by the WinCC OA GraphQL & REST API Server.

The API server supports the following authentication methods:

- **Username/password authentication** – Users authenticate using credentials configured in the .env file or the WinCC OA user database.
- **Direct access tokens** – Predefined static access tokens configured in the .env file.

Authenticated API requests use bearer authentication via the HTTP Authorization header.

Example:

Authorization: Bearer <token>

Protected API requests without a valid authentication token are rejected.

3.1. JWT Authentication Workflow

When username/password authentication is used, the server generates a JSON Web Token (JWT) after successful credential validation.

Example .env configuration:

```
ADMINUSERNAME=<admin_username>
ADMIN_PASSWORD=<strong_password>
```

The generated JWT token must be included in authenticated API requests using the HTTP Authorization header:

Authorization: Bearer <jwt_token>

JWT tokens:

- are generated dynamically after successful authentication,
- contain user and authorization information,
- and expire automatically after the configured validity period.

The token lifetime is configured using:

```
TOKEN_EXPIRY_MS=3600000
```

3.2. Direct Access Tokens

The API server optionally supports predefined access tokens configured in the .env file.

Example configuration:

```
DIRECT_ACCESS_TOKEN=my_admin_token
READONLY_TOKEN=my_readonly_token
```

- **DIRECT_ACCESS_TOKEN** – Predefined token with administrative access rights
- **READONLY_TOKEN** – Predefined token with read-only access rightsThe tokens are used in authenticated API requests via the HTTP Authorization header:

Authorization: Bearer <token>

Direct access tokens allow authenticated API access without executing a login request.

Security Note:

Direct access tokens provide authenticated API access without requiring a login request. These tokens should therefore be protected against unauthorized access and should not be exposed publicly or stored in unsecured locations.

3.3. Authorization Concepts

The server supports role-based authorization with **administrative** and **read-only** access levels.
Administrative users are permitted to execute GraphQL mutations, perform REST write operations, create datapoints, and modify datapoint values.
Read-only users are restricted to GraphQL query operations, REST read operations, and monitoring functionality.
Write operations requested by read-only users are rejected by the authorization layer.
Authentication and authorization behavior is configured through the .env file.

```
AUTH_MODE=config
DISABLE_AUTH=false

ADMIN_USERNAME=example_admin
ADMIN_PASSWORD=strong_password

READONLY_USERNAME=example_readonly
READONLY_PASSWORD=strong_password
```

Security Note:
The credentials shown above are example values for demonstration purposes only.
Do not use default or example credentials in productive environments. Replace all passwords, secrets, and access tokens with strong project-specific values.

- AUTH_MODE=config uses the users defined in the .env file.
- AUTH_MODE=winccoa uses the WinCC OA user database.
- AUTH_MODE=both first checks the .env users and then falls back to the WinCC OA user database.
- DISABLE_AUTH=false enables authentication.
- ADMIN_USERNAME / ADMIN_PASSWORD define administrative access.
- READONLY_USERNAME / READONLY_PASSWORD define read-only access.

4. Operation

4.1. GraphQL API

4.1.1. Apollo Sandbox Overview

The GraphQL endpoint provides the Apollo Sandbox interface for testing, validating, and executing GraphQL operations directly in the browser.
Apollo Sandbox provides the following features:

- a GraphQL operation editor,
- an interactive schema browser,
- query execution and response visualization,
- and support for authenticated requests and subscriptions.

The interface also supports multiple GraphQL operation tabs that can be executed independently within the same session.
Additional information about Apollo Sandbox is available in the official Apollo documentation: Apollo Sandbox Documentation

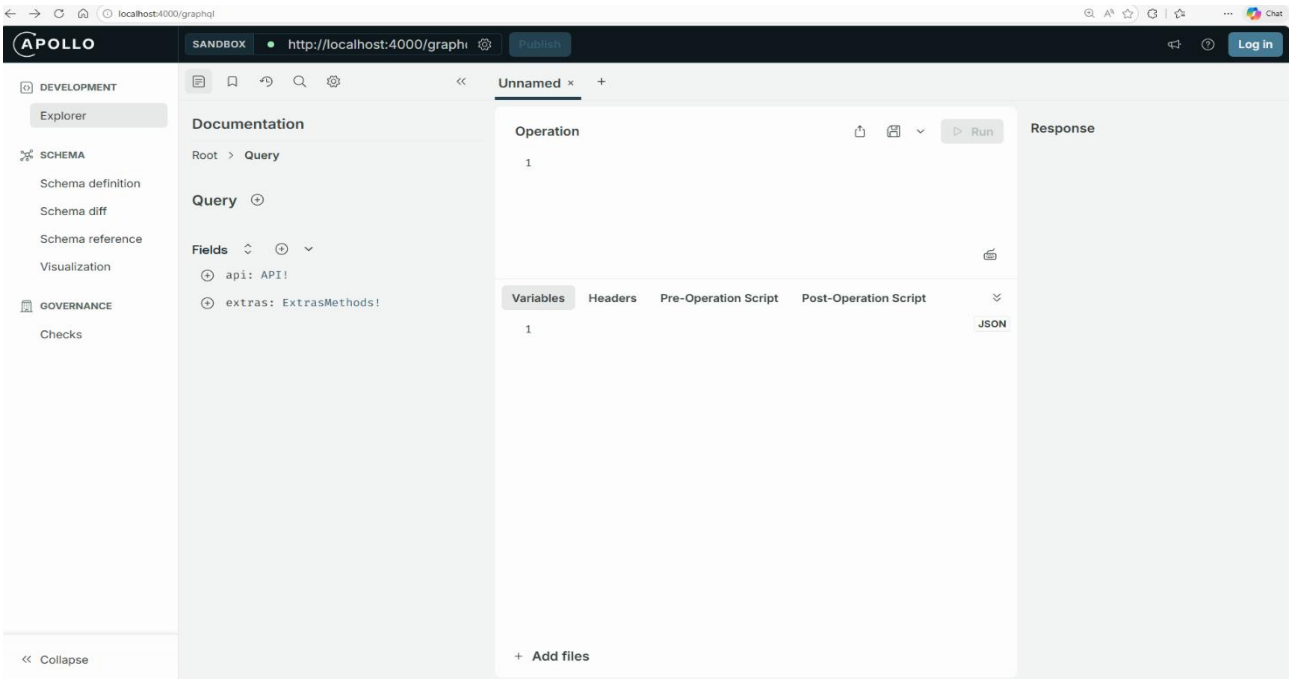


Figure 4-1 Apollo Sandbox user interface overview

4.1.2. GraphQL Authentication

The GraphQL API supports authenticated queries, mutations, and subscriptions.

When username/password authentication is used, the server returns a JWT access token after successful authentication.

Example login mutation:

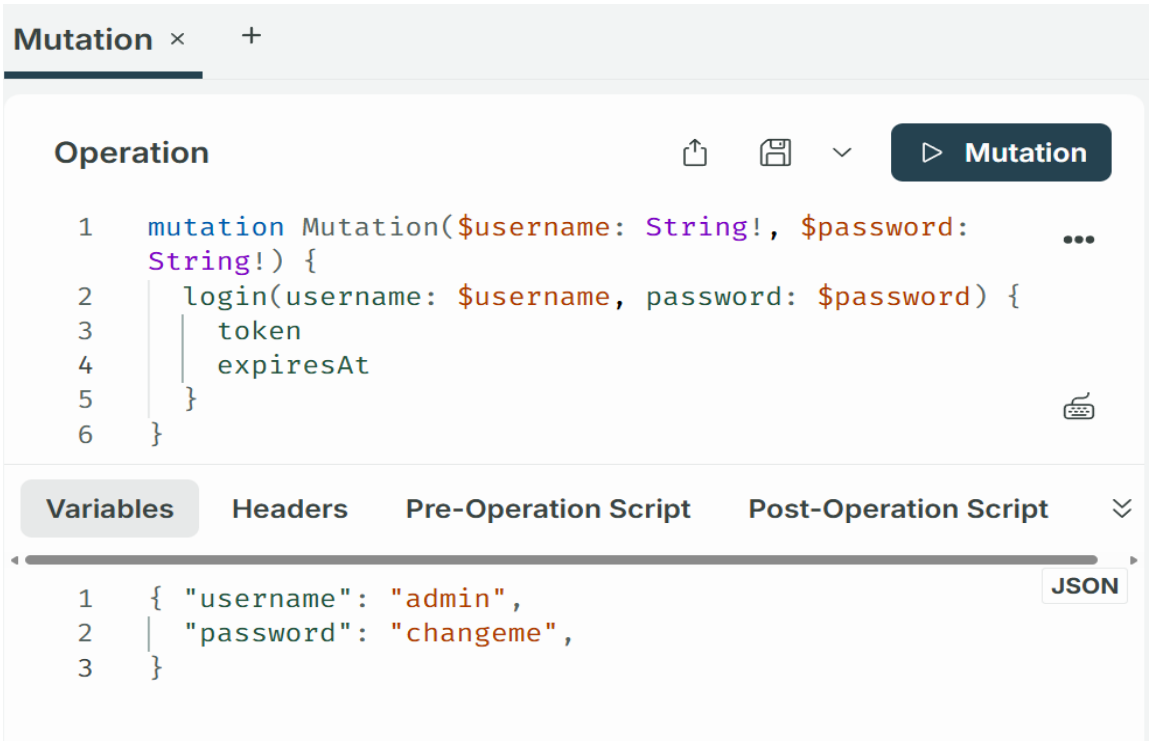


Figure 4-2 Executing the GraphQL login mutation in Apollo Sandbox

```
{
  "data": {
    "login": {
      "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
      eyJ1c2VySWQiOiJhZG1pbiIsInRva2VuSWQiOiIwYTA3OTk4MS1lMDFhLTRlNzAtYjcyMi0yMWJkY
      jk1M2I4MWMiLCJleHBpcmVzQXQiOjE3Nzg0MzI0NDAwMjYsInJvbGUiOiJhZG1pbiIsImlhdCI6MT
      c3ODQyODg0MCwiZXhwIjoxNzc4NDMyNDQwfQ.
      e9WF6r0eafhtCUjHz4nmJahbemHCk8ABKVIZmKp1pys",
      "expiresAt": "2026-05-10T17:00:40.026Z"
    }
  }
}
```


Figure 4-3 successful JWT authentication response returned by the GraphQL API

The returned JWT token is used for authenticated GraphQL requests via the HTTP Authorization header: Authorization: Bearer <token>.

To configure authenticated GraphQL requests in Apollo Sandbox:

1. Open the connection settings using the settings icon next to the GraphQL endpoint.

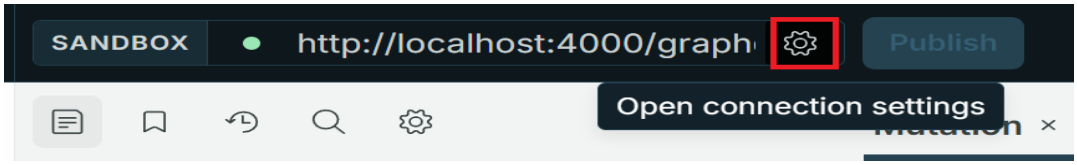


Figure 4-4 Opening the Apollo Sandbox connection settings

2. Add a shared header with:
    - Header key: Authorization
    - Header value: Bearer <token>

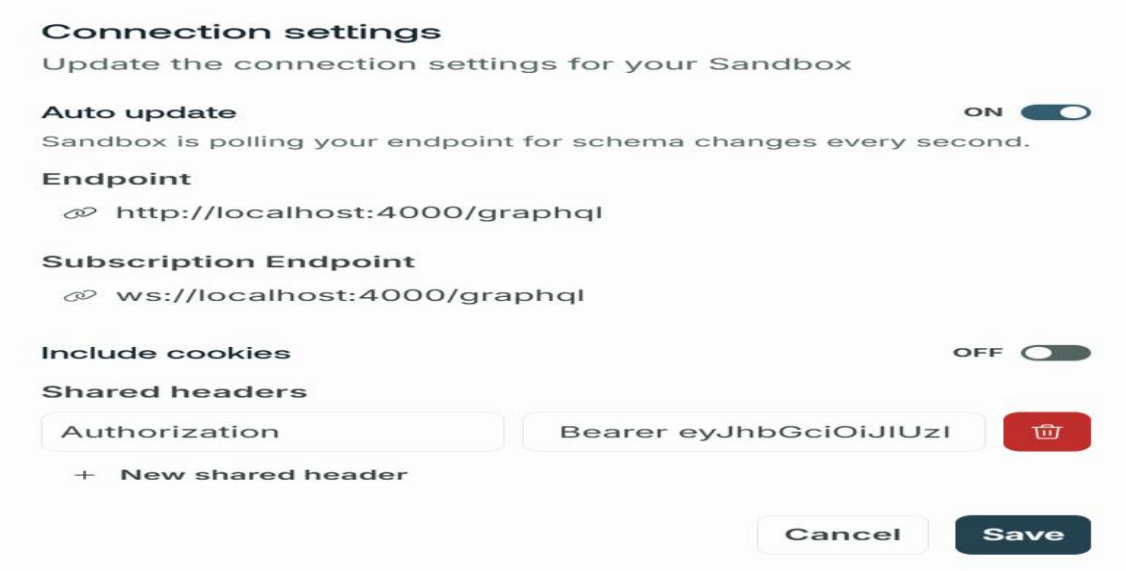


Figure 4-5 Configuring JWT authentication for GraphQL requests in Apollo Sandbox

3. Save the connection settings.

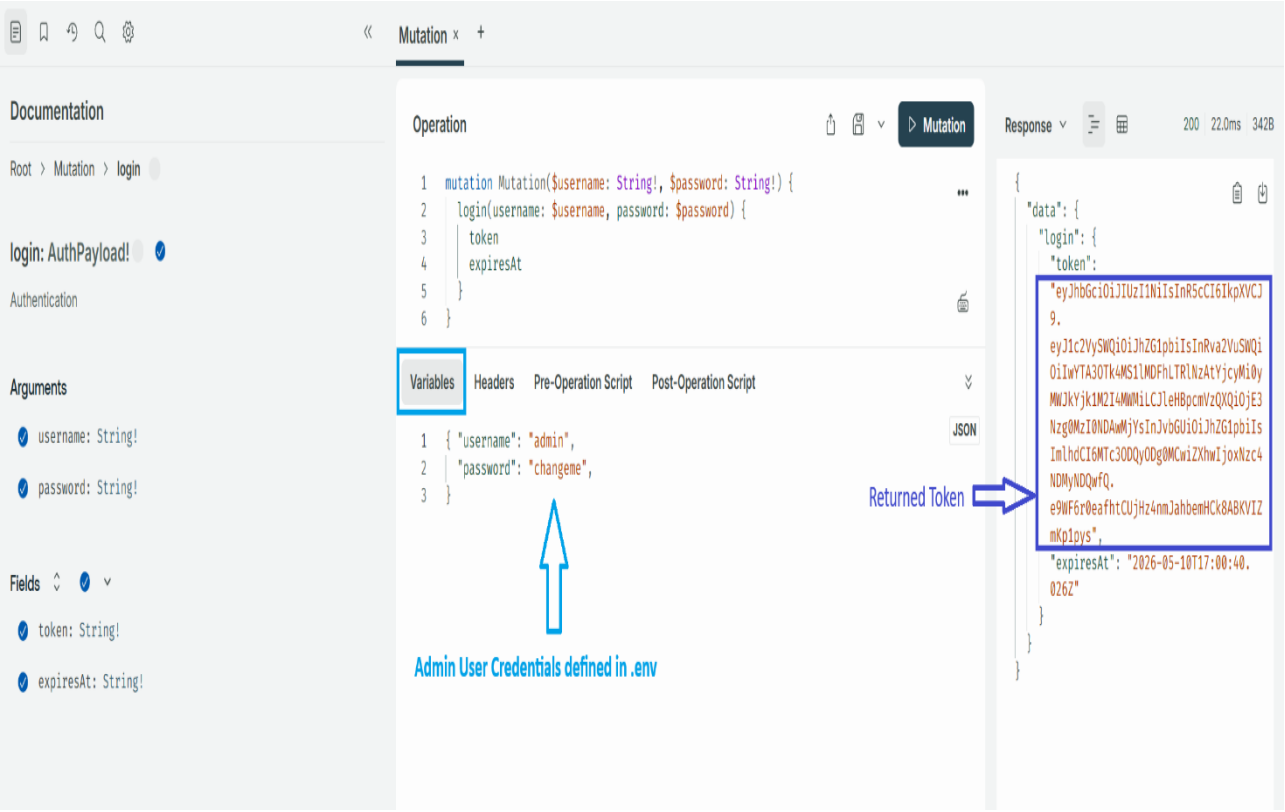


Figure 4-6 Executing authenticated GraphQL requests using the configured JWT token

After successful authentication configuration, GraphQL queries, mutations, and subscriptions are executed using the configured JWT token in this example.

4.1.3. Exploring the GraphQL Schema

The GraphQL schema defines the available WinCC OA API operations and their corresponding input and output types.
The schema provides the following root operation types:

Root Type	Purpose
query	Reads data from WinCC OA
mutation	Creates or modifies data
subscription	Receives real-time updates

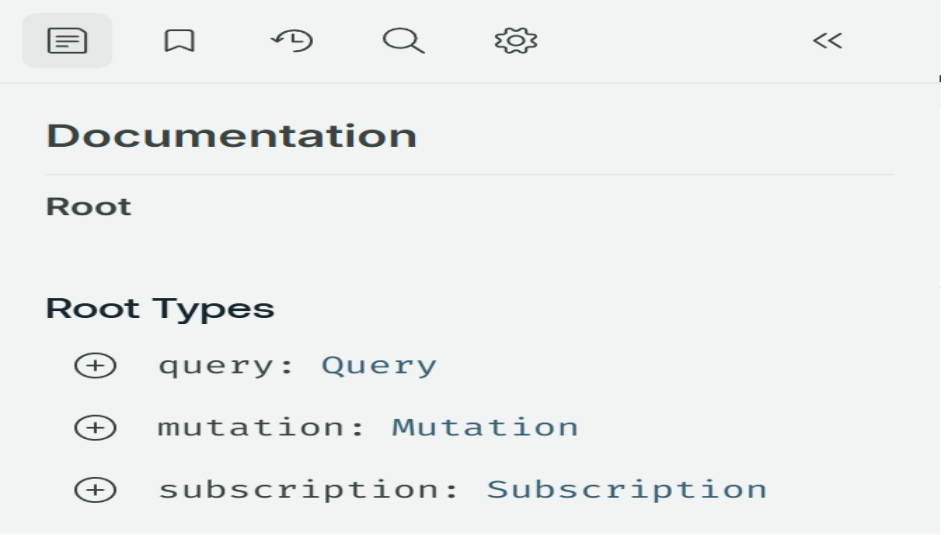


Figure 4-7 GraphQL root operation types displayed in Apollo Sandbox

Available functions and arguments can be explored interactively using the schema browser in Apollo Sandbox.

Functions requiring input arguments provide:

- argument definitions,
- GraphQL type information,
- return types,
- and operation documentation.

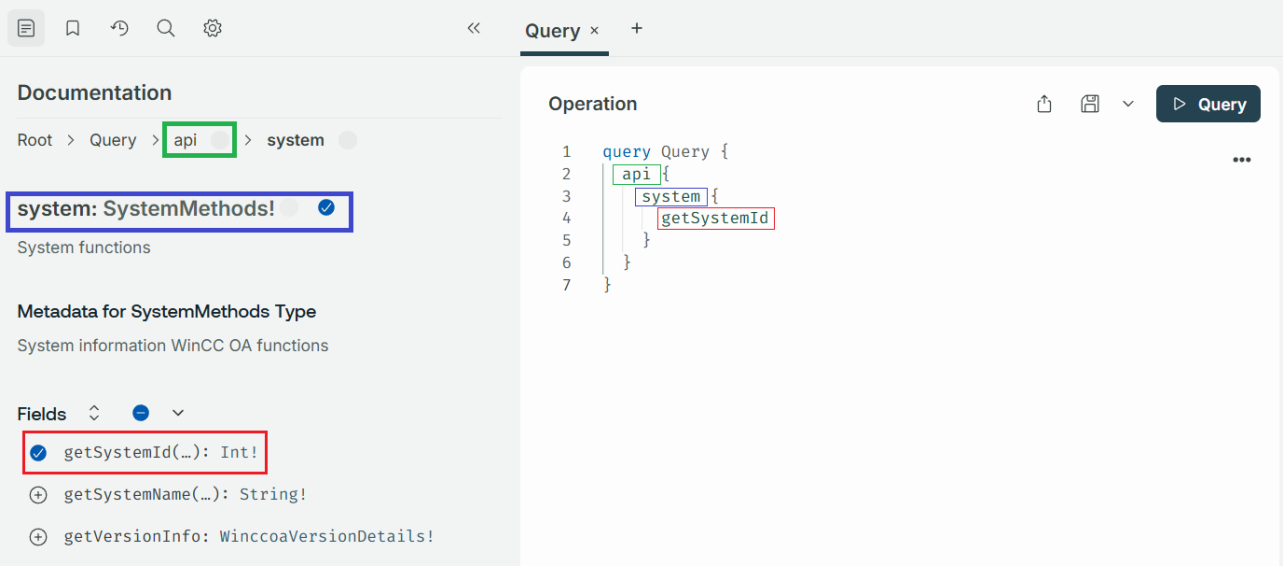


Figure 4-8 Building a GraphQL query using the Apollo Sandbox schema browser

4.1.4. Executing GraphQL Queries

3.1.4.1. Executing Basic Queries

GraphQL queries are used to read data from the WinCC OA system.

Queries can be executed directly in Apollo Sandbox using the GraphQL operation editor. The response is returned in JSON format in the response panel.

The following example retrieves the WinCC OA system ID and system name:

A new GraphQL operation tab can be created by clicking the **plus (+)** button above the GraphQL operation editor. This allows multiple queries, mutations, or subscriptions to be tested independently within the same session.

A new GraphQL operation tab can be created by clicking the plus (+) button above the GraphQL operation editor. This allows multiple queries, mutations, or subscriptions to be tested independently within the same session.

Example Query

```
query {
  api {
    system {
      getVersionInfo
    }
  }
}
```

Figure 4-9 Building and executing a GraphQL query using Apollo Sandbox

Returned response example:

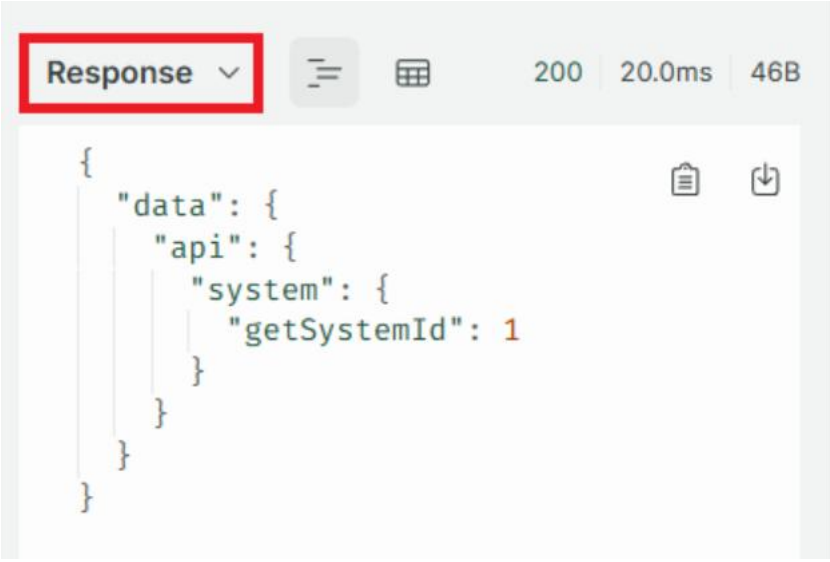


Figure 4-10 JSON response returned by the GraphQL query

3.1.4.2. Executing Parameterized Queries Using Variables

Functions requiring input arguments can be executed using GraphQL variables.

The following example uses the dp.get function to retrieve the current value of a datapoint element:

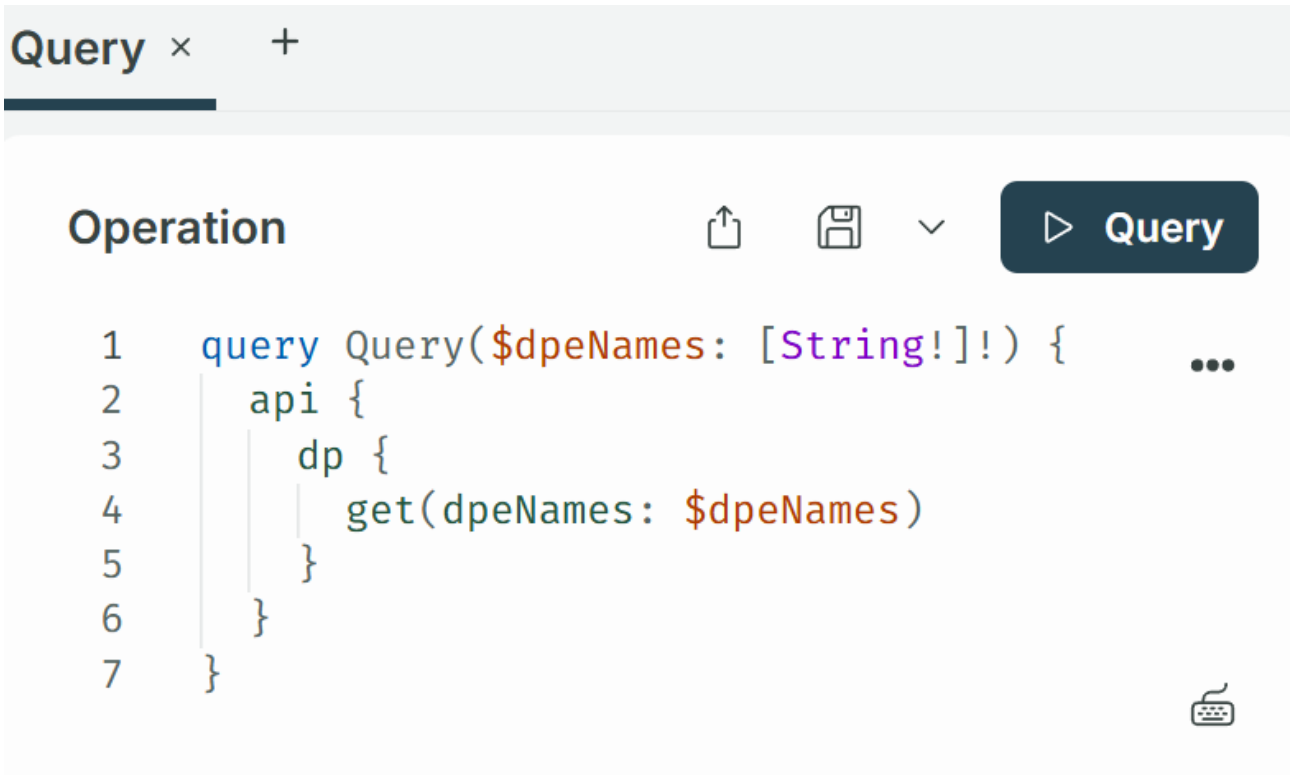


Figure 4-11 Executing a parameterized GraphQL query using variables

Variables:

{

"dpeNames": [

"ExampleDP_Result.:_original.._value"

]

}

Figure 4-12 Defining GraphQL query variables in Apollo Sandbox

Returned response example:

Figure 4-13 JSON response returned by the parameterized GraphQL query

The variable values are defined in the Variables section below the GraphQL editor.
The specified datapoint element corresponds to the _original.._value attribute of the WinCC OA datapoint ExampleDP_Arg1.

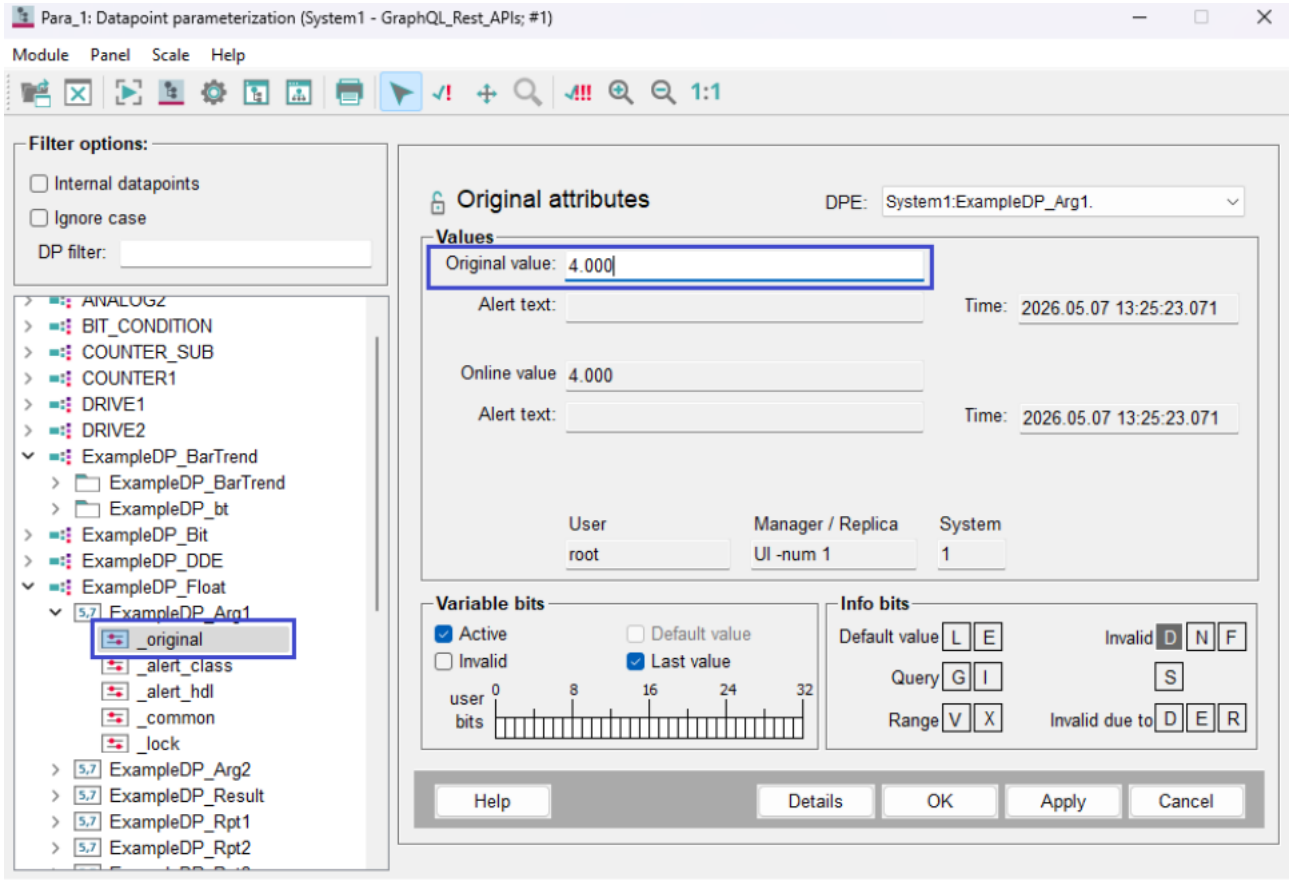


Figure 4-14 WinCC OA PARA view of the queried datapoint element

4.1.5. Executing GraphQL Mutations

GraphQL mutations are used to create, modify, or delete data in the WinCC OA project.

Mutations can be executed directly in Apollo Sandbox using the GraphQL operation editor.

The following example demonstrates how to create a new datapoint using the create mutation:

```
mutation($dpType: String!, $dpName: String!) {
  api {
    dp {
      create(type: $dpType, dps: [$dpName])
    }
  }
}
```

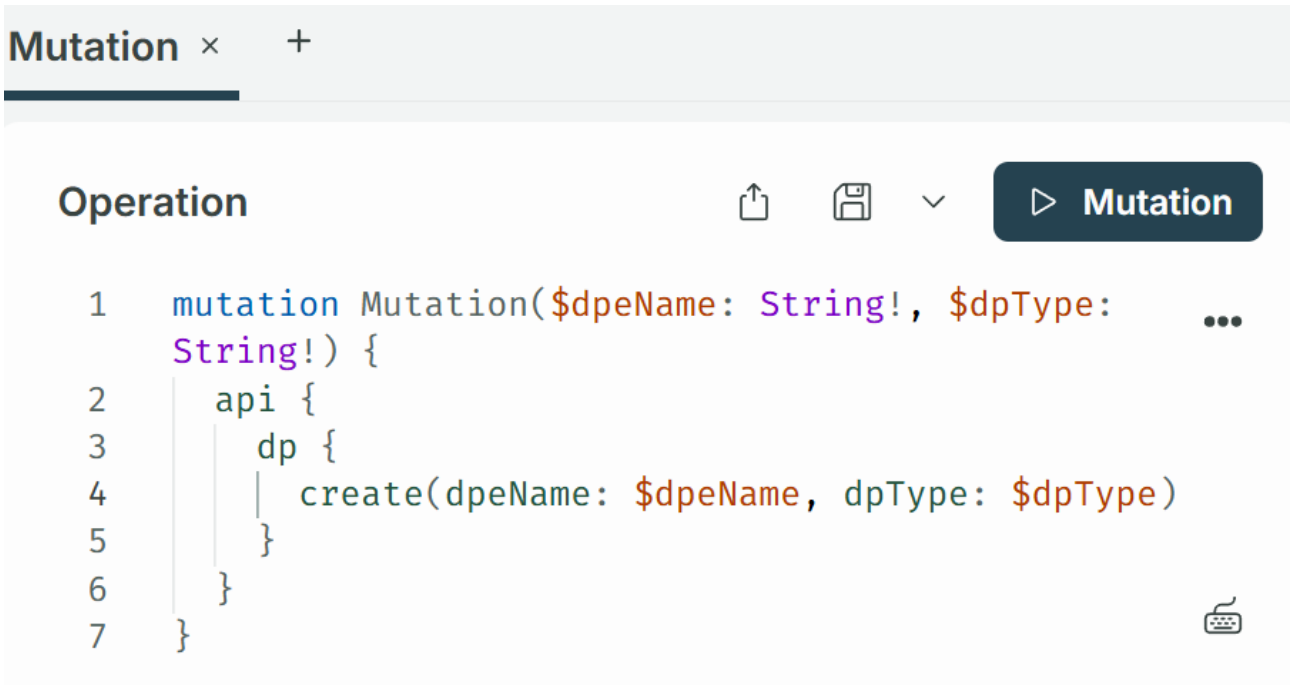


Figure 4-15 Executing a GraphQL mutation using Apollo Sandbox

Variables:

```
{
 "dpType": "ExampleDP_Bit",
 "dpName": "ExampleDP_New"
}
```

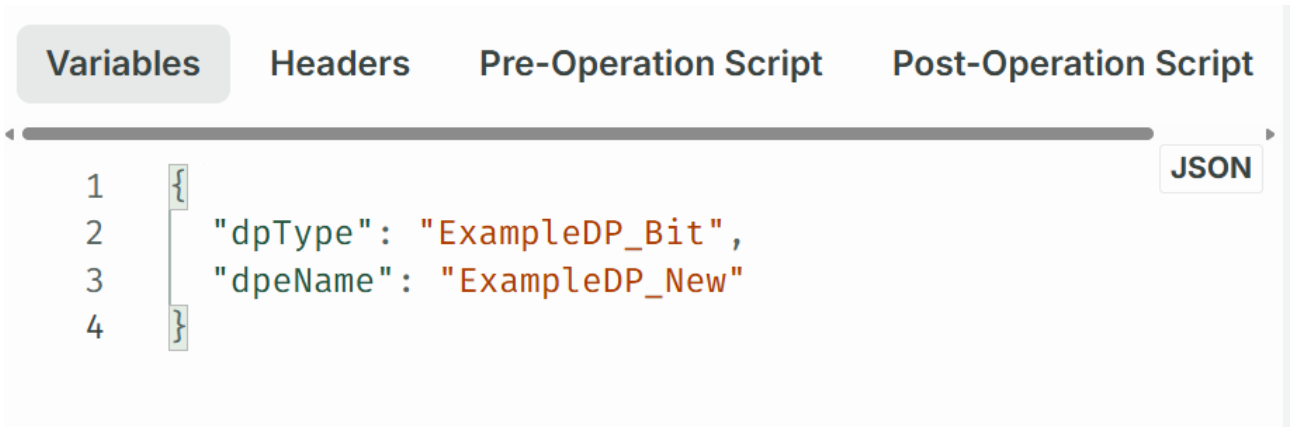


Figure 4-16 Defining variables for the GraphQL mutation

Returned response example:

Figure 4-17 Successful response returned by the GraphQL create mutation

The returned value true confirms that the mutation was executed successfully and that the datapoint was created in the WinCC OA project.

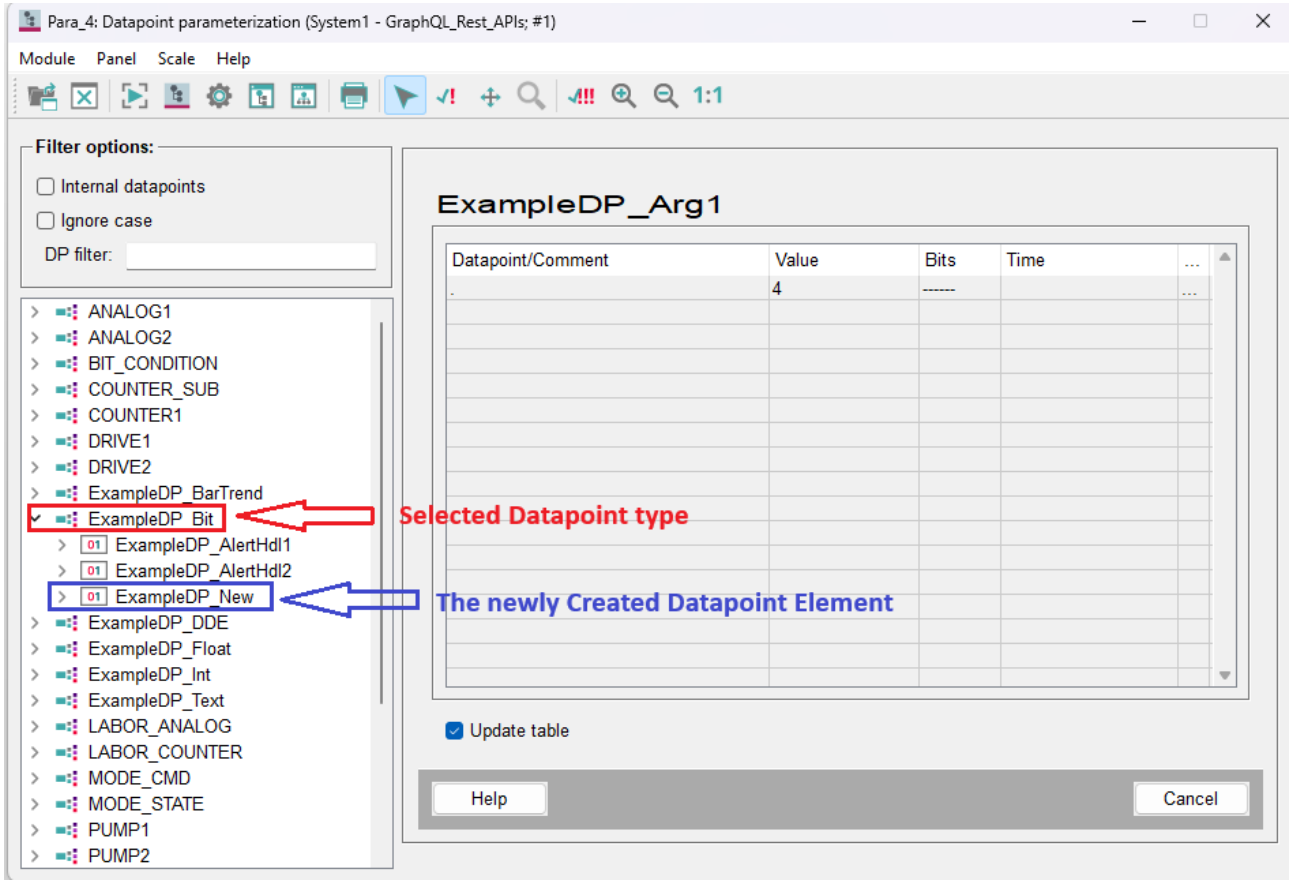


Figure 4-18 Created datapoint displayed in the WinCC OA PARA module

4.1.6. Executing GraphQL Subscriptions

GraphQL subscriptions are used to receive real-time updates from the WinCC OA system.

Subscriptions maintain an active WebSocket connection and automatically return updated values whenever monitored datapoint values change.

For testing subscriptions, the Altair GraphQL Client is used in this example.

Example subscription using dpQueryConnectSingle:

```
subscription DpQueryConnectSingle($query: String!) {
 dpQueryConnectSingle(query: $query) {
  values
  type
  error
 }}
```

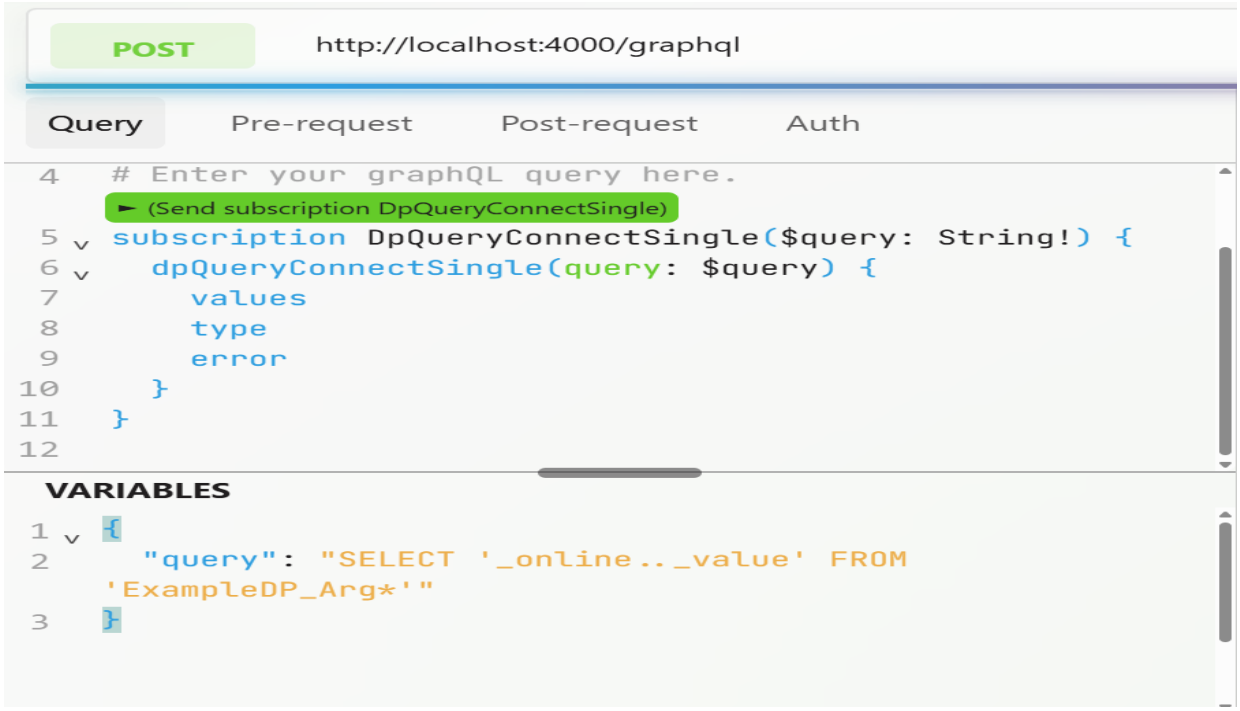


Figure 4-19 Executing the dpQueryConnectSingle GraphQL subscription in Altair GraphQL Client

Client Subscription updates are displayed continuously while the WebSocket connection remains active.

Figure 4-20 Real-time updates received from the dpQueryConnectSingle GraphQL subscription

4.2. REST API

This chapter describes the REST API interface and the integrated Swagger/OpenAPI documentation provided by the WinCC OA GraphQL & REST API Server.

The REST API endpoint is available at:

http://localhost:4000/restapi

The interactive Swagger/OpenAPI documentation is available at:

http://localhost:4000/api-docs

4.2.1. REST API Overview

The REST API provides HTTP-based access to WinCC OA functionality through dedicated REST endpoints.
Supported HTTP methods include:

- GET for read operations,
- POST for create and execution operations,
- PUT for update operations,

- DELETE for delete operations.

The REST API supports operations for:

- datapoint management,
- alert handling,
- CNS operations,
- system information queries,
- authentication,
- and health monitoring.

All REST API responses are returned in JSON format.

4.2.2. REST API Authentication

The REST API supports authenticated requests using the HTTP Authorization header.

When username/password authentication is used, a JWT token can be obtained using the authentication endpoint:

POST http://localhost:4000/restapi/v1/auth/login

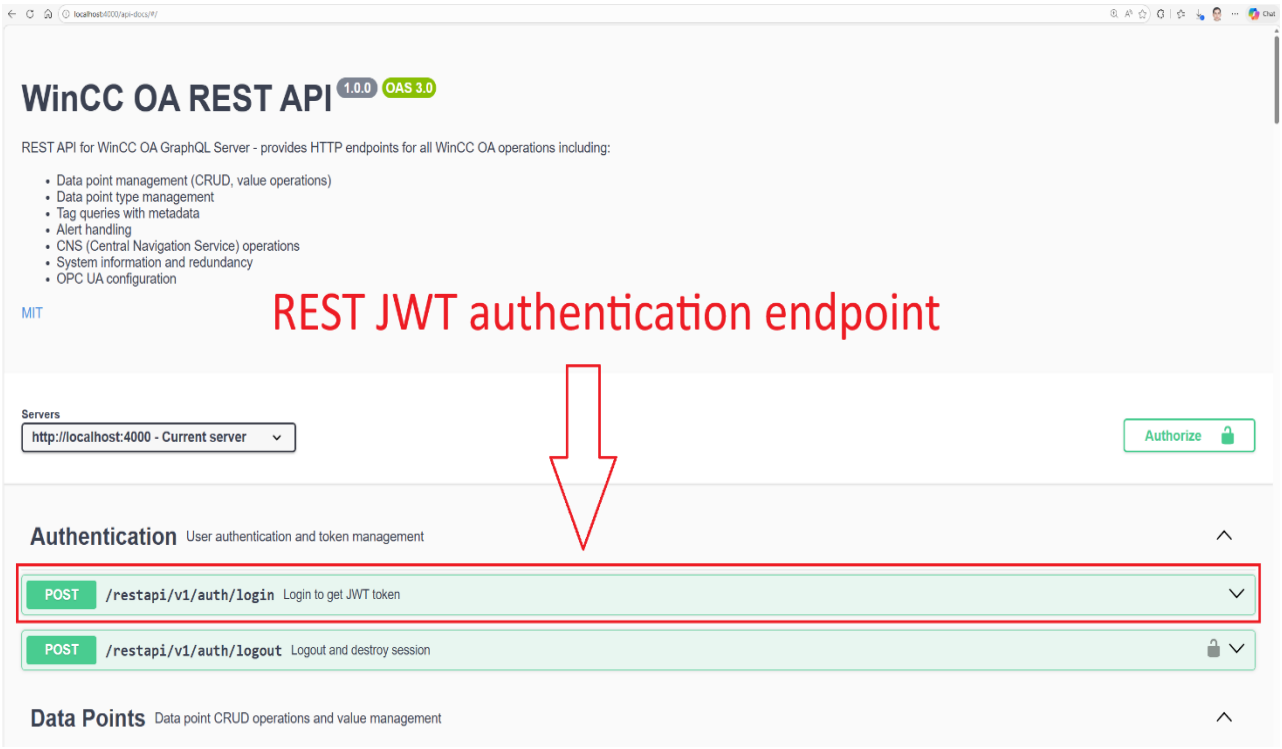


Figure 4-21 REST JWT authentication endpoint in Swagger UI

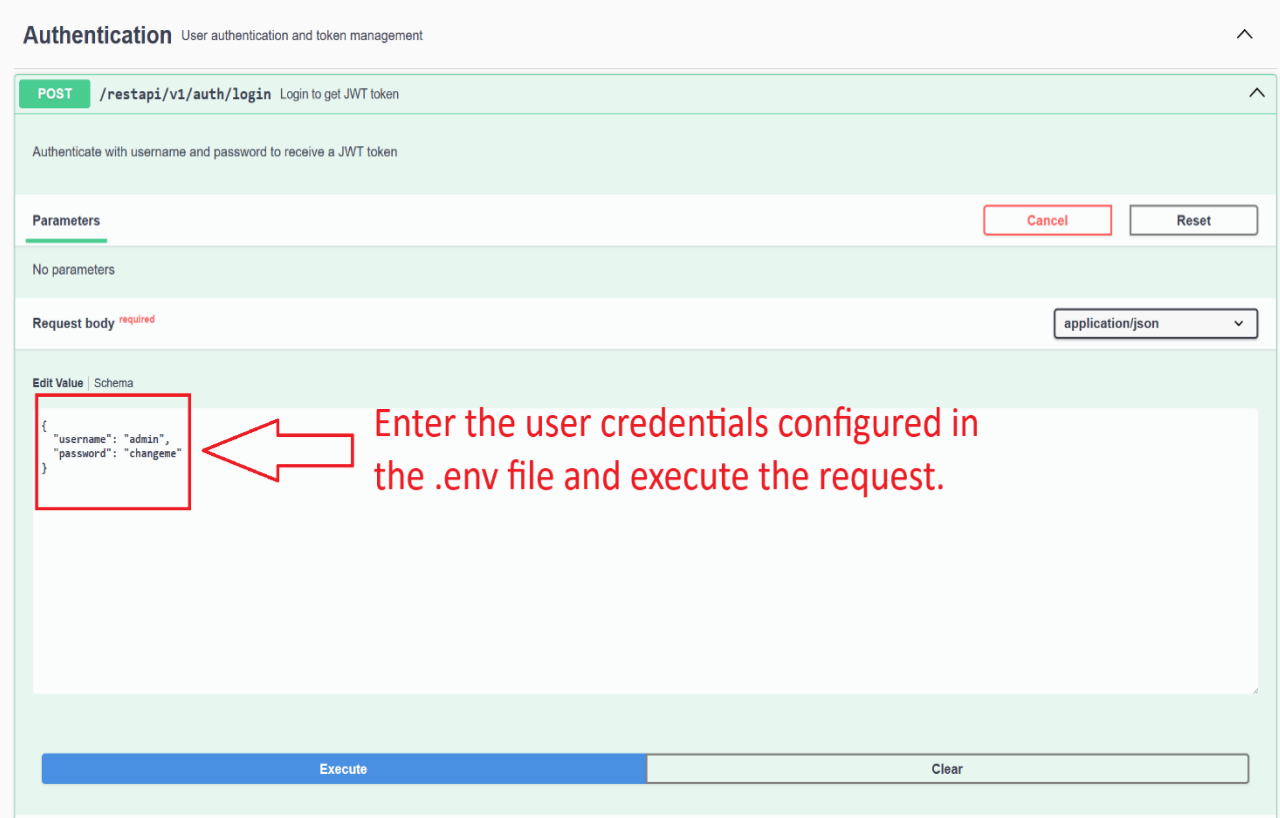


Figure 4-22 Executing the REST authentication request in Swagger UI

After successful authentication, the server returns:

- a JWT access token,
- and its expiration time.

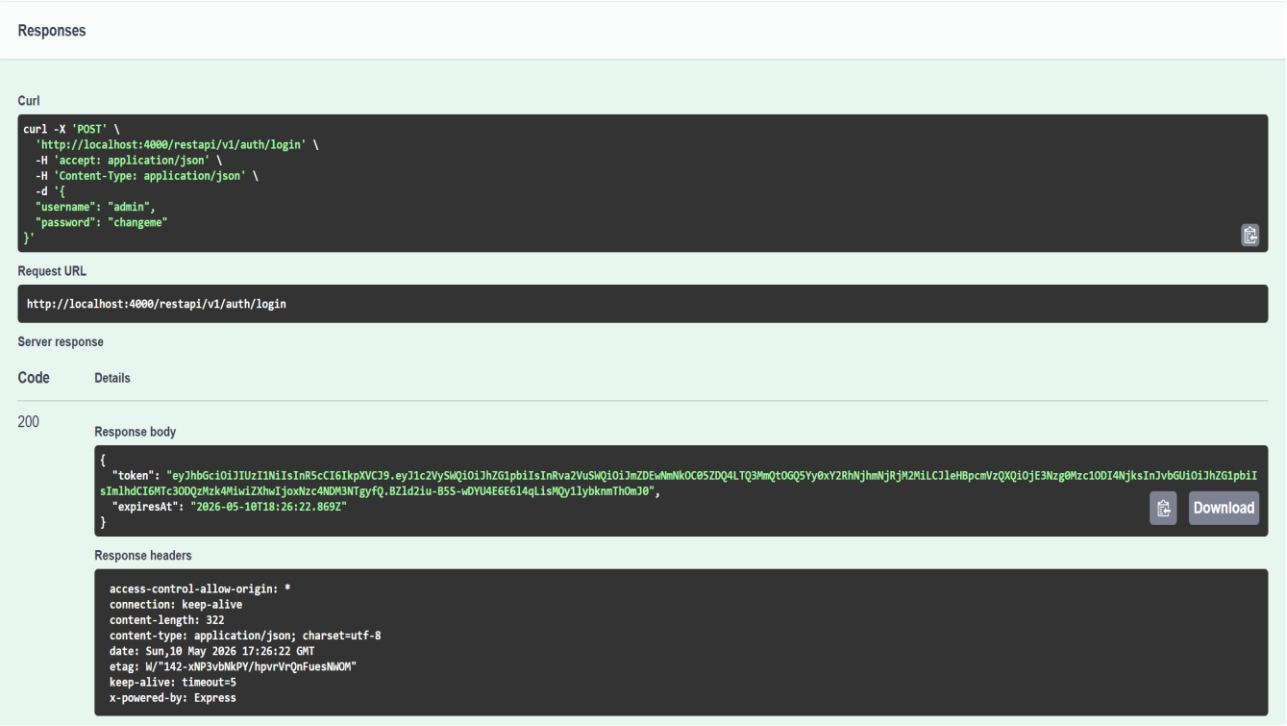


Figure 4-23 JWT token returned by the REST authentication endpoint

The returned token must be included in subsequent REST API requests using the HTTP Authorization header:

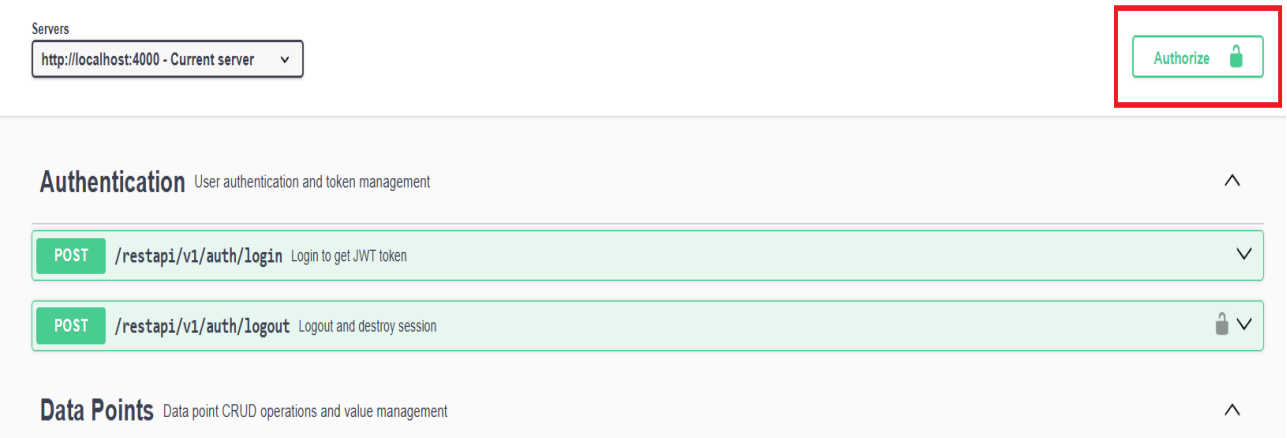


Figure 4-24 Opening the Swagger UI authorization configuration

Authenticated REST requests use the HTTP Authorization header:

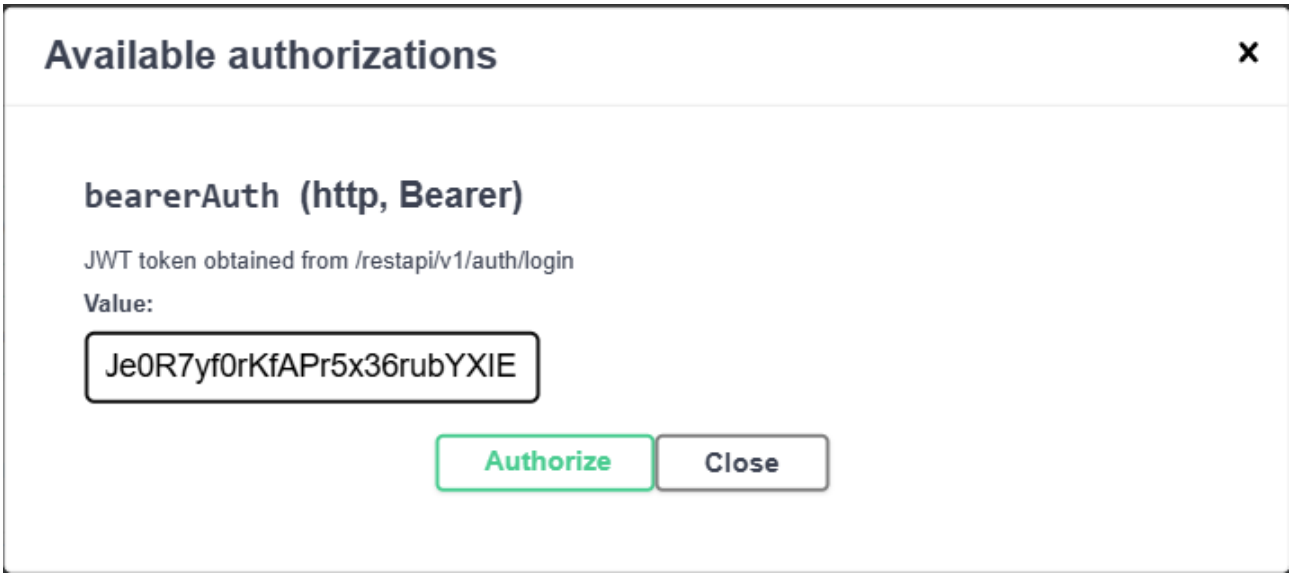


Figure 4-25 Configuring the Authorization header for authenticated REST API requests

Curl

```
curl -X 'POST' \
  'http://localhost:4000/restapi/v1/auth/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "username": "admin",
  "password": "changeme"
}'
```

Figure 4-26 Executing authenticated REST API requests in Swagger UI

The authentication token can be either:

- a JWT token returned by the authentication endpoint,
- or a predefined direct access token.

4.2.3. Executing REST Requests

Swagger UI provides an overview of the available REST API endpoints grouped by functional categories, including:

- Authentication
- Data Points
- Data Point Types
- Tags
- Alerts
- CNS
- System
- Subscriptions

Each endpoint displays:

- the HTTP method,
- endpoint URL,
- request parameters,
- response schemas,
- and response codes.

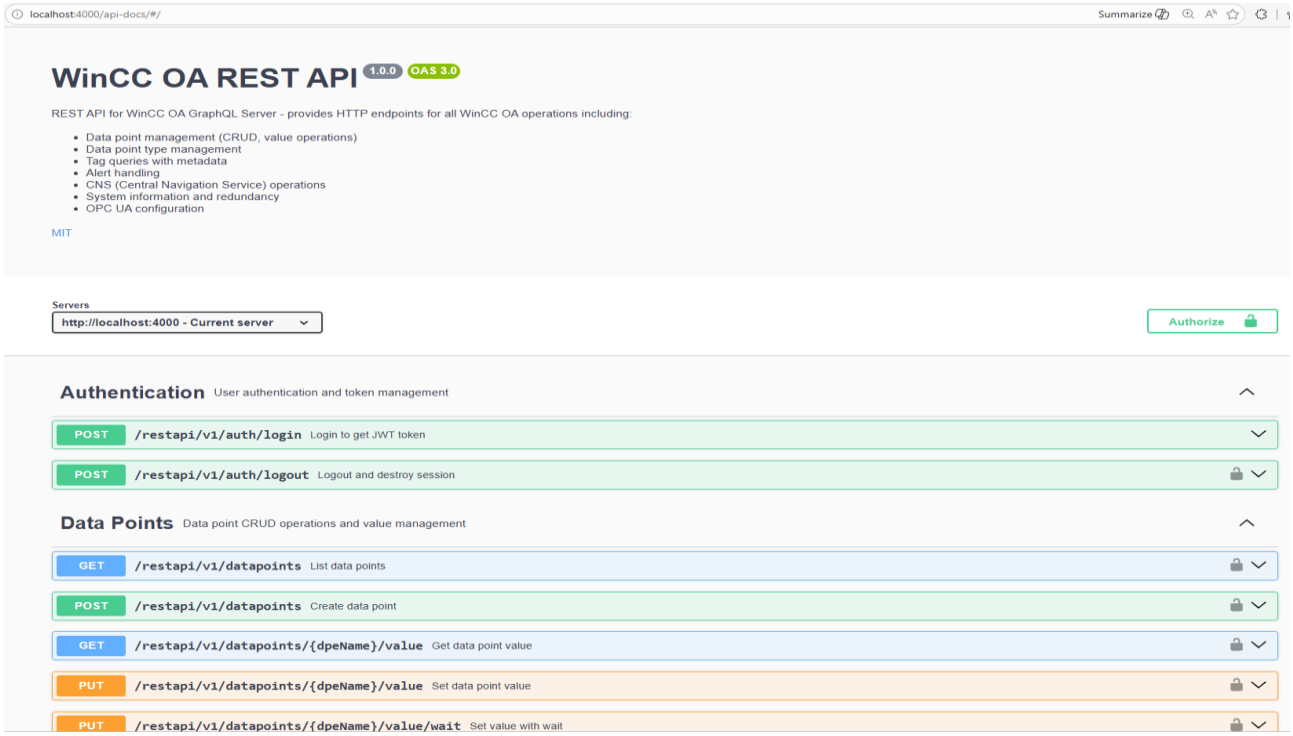


Figure 4-27 REST API endpoint categories in Swagger UI

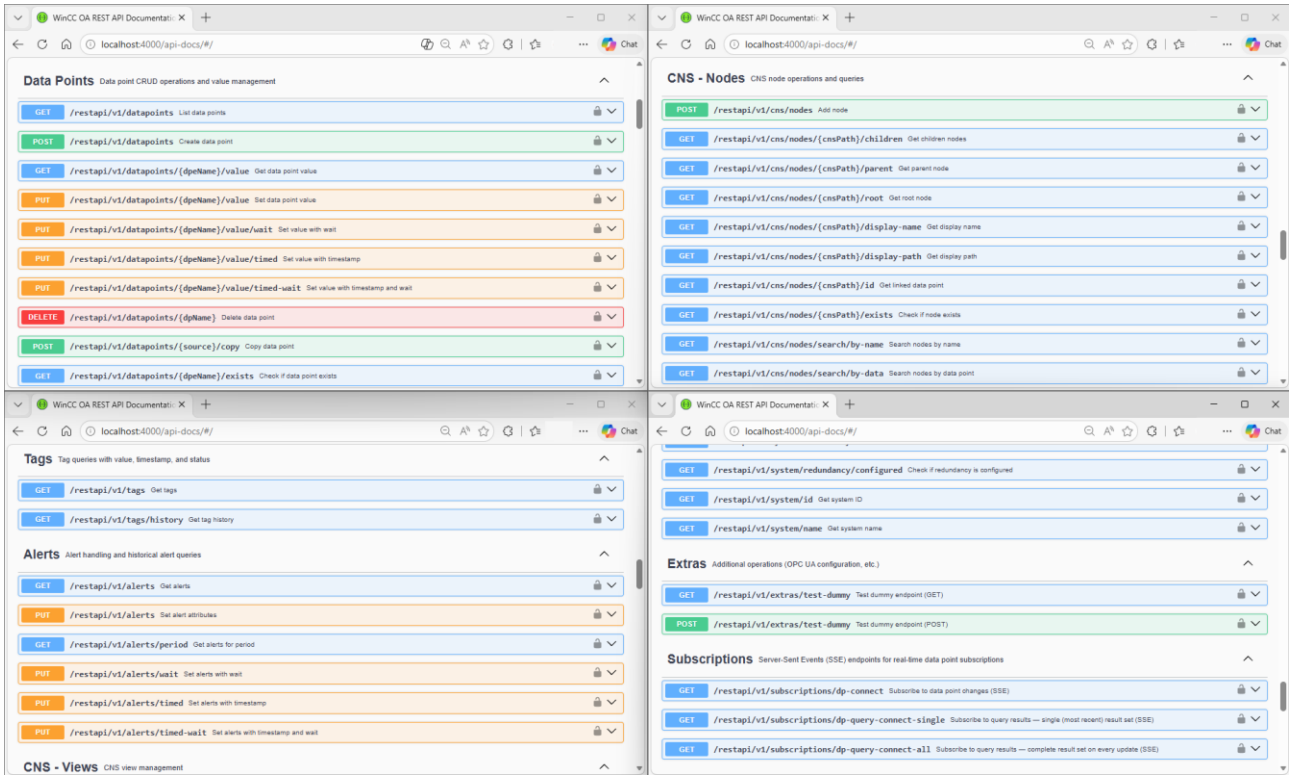


Figure 4-28 REST API endpoints for datapoint, alert, and CNS operations

The following example demonstrates creating a CNS view using the REST API Endpoint:

POST /restapi/v1/cns/views

Example request body:

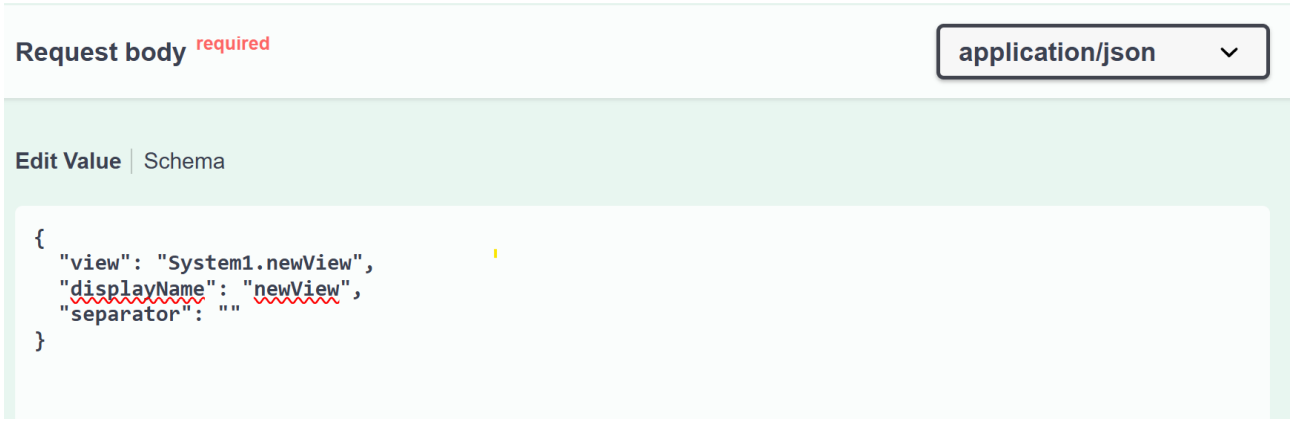


Figure 4-29 Example request body for creating a CNS view in Swagger UI

Example response:

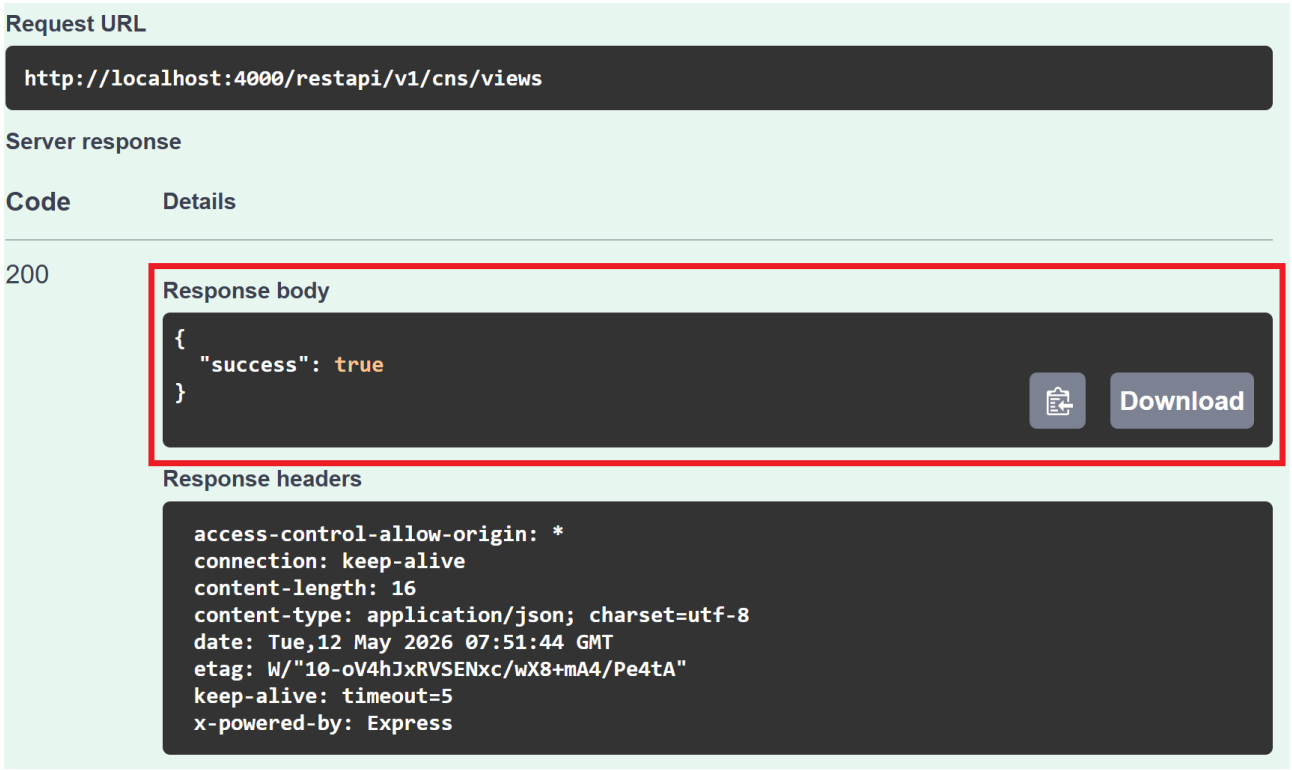


Figure 4-30 Successful response returned after creating a CNS view

3.1.4.1. Example – Creating a CNS View

The following example creates a CNS view using the REST API.

Endpoint: POST /restapi/v1/cns/views

Example request body:

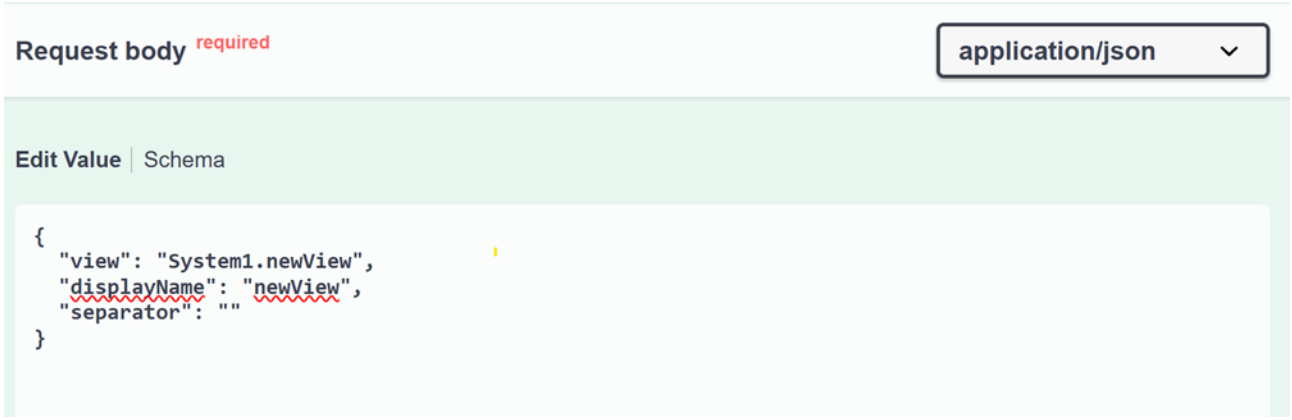


Figure 7-1 Configuring the REST request body for CNS view creation

Example response:

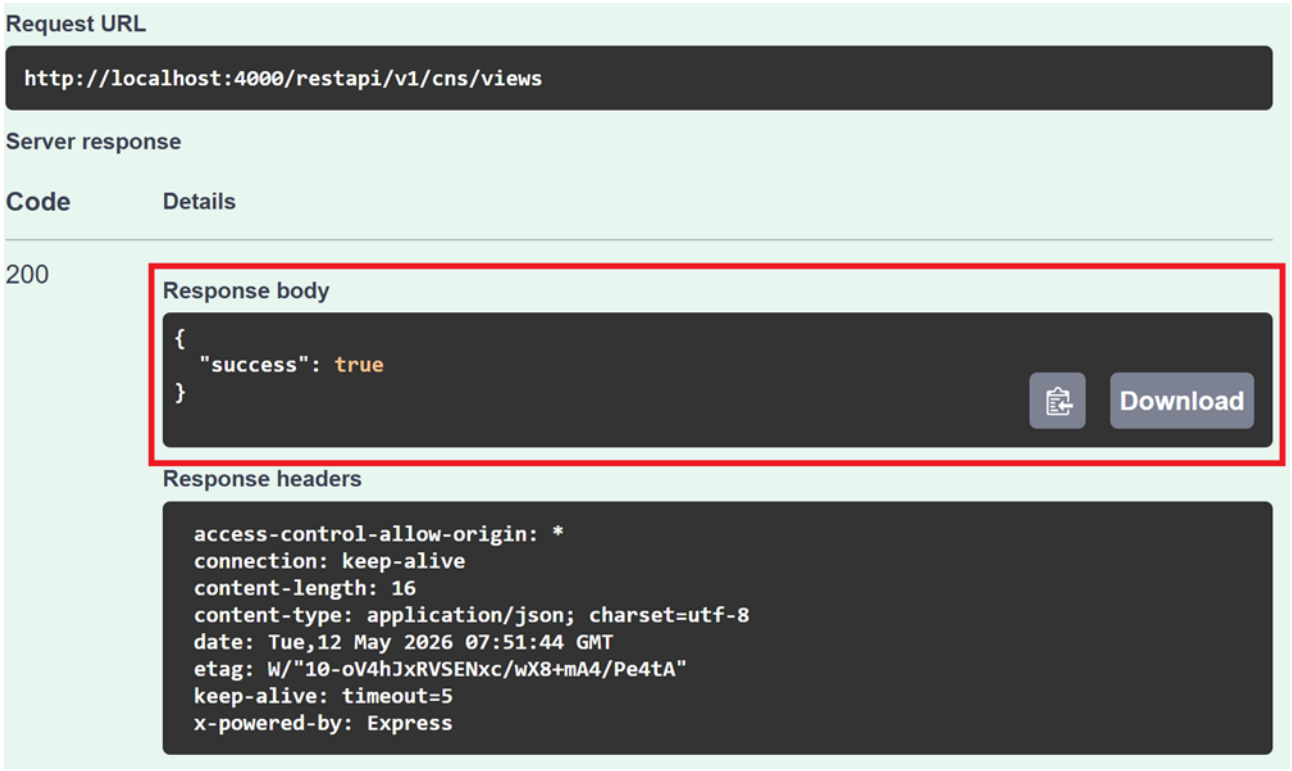


Figure 7-2 Creating a CNS view using Swagger UI

3.1.4.2. Example – Creating a CNS Tree

After creating a CNS view, CNS trees can be added to the view.

Endpoint: POST /restapi/v1/cns/trees

Example request body:

The complete CNS tree example request body is available in: cns_tree_example.json

Example response:

Figure 4-31 Creating a CNS tree using Swagger UI

The request creates a hierarchical CNS tree under the CNS view `**System1.newView**`.
Each node contains a unique identifier (name), multilingual display names (displayName), and optional datapoint references (dp).

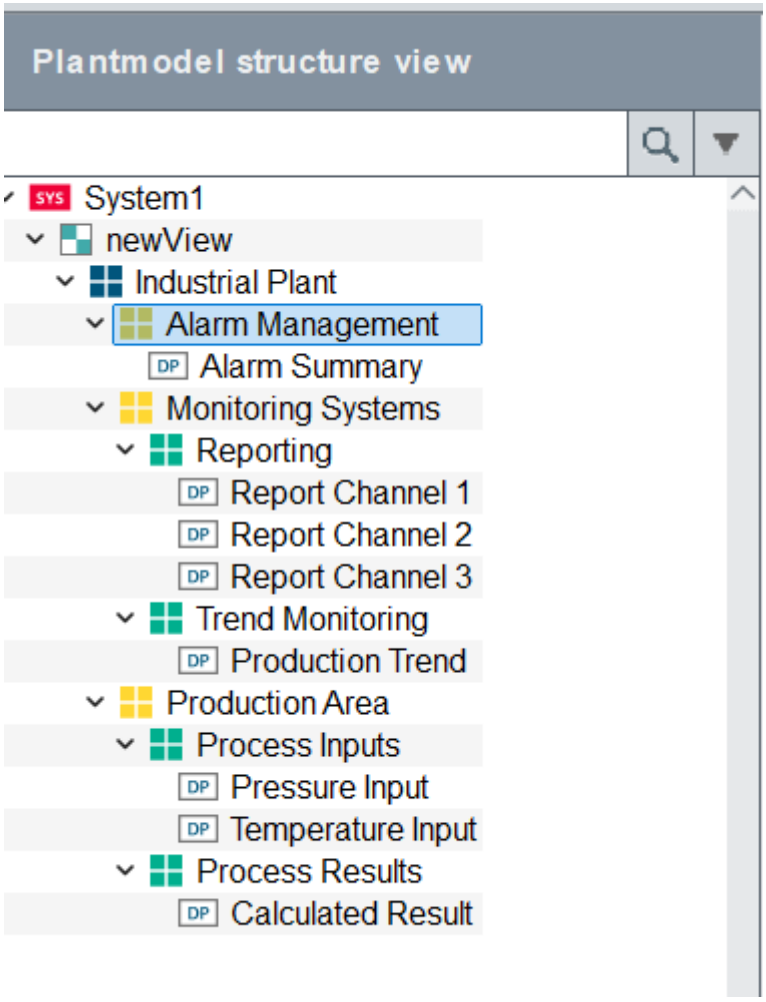


Figure 4-32 CNS tree created using the REST API displayed in the WinCC OA Plantmodel Editor

4.2.4. OpenAPI Specification

The OpenAPI specification is available at:

http://localhost:4000/openapi.json

The OpenAPI specification describes the REST API endpoints exposed by the server and is used by Swagger UI to generate the interactive REST API documentation.

The specification follows the OpenAPI 3.0 standard and contains:

- endpoint definitions,
- request schemas,
- response schemas,
- authentication definitions,
- parameter descriptions,
- and API metadata.

The specification can also be imported into external tools such as:

- Postman,
- API testing tools,
- SDK generators,

and integration frameworks

4.3. API Usage Statistics

The API server provides a statistics page for monitoring GraphQL and REST API activity during runtime.

The statistics page is available directly from the landing page under **Usage Statistics** or via:

http://localhost:4000/stats.html

The statistics page includes:

- total GraphQL and REST API request counters,
- graphical API usage visualization,
- per-endpoint statistics,
- filtering and sorting functions,
- regular expression (Regex) filtering,
- manual refresh of runtime statistics,
- and exclusion of internally generated /stats requests from the displayed counters.

Excluding internally generated /stats requests prevents self-tracking and distorted statistics counters.

The displayed statistics are updated during runtime and can be used for monitoring, testing, debugging, and API usage analysis.

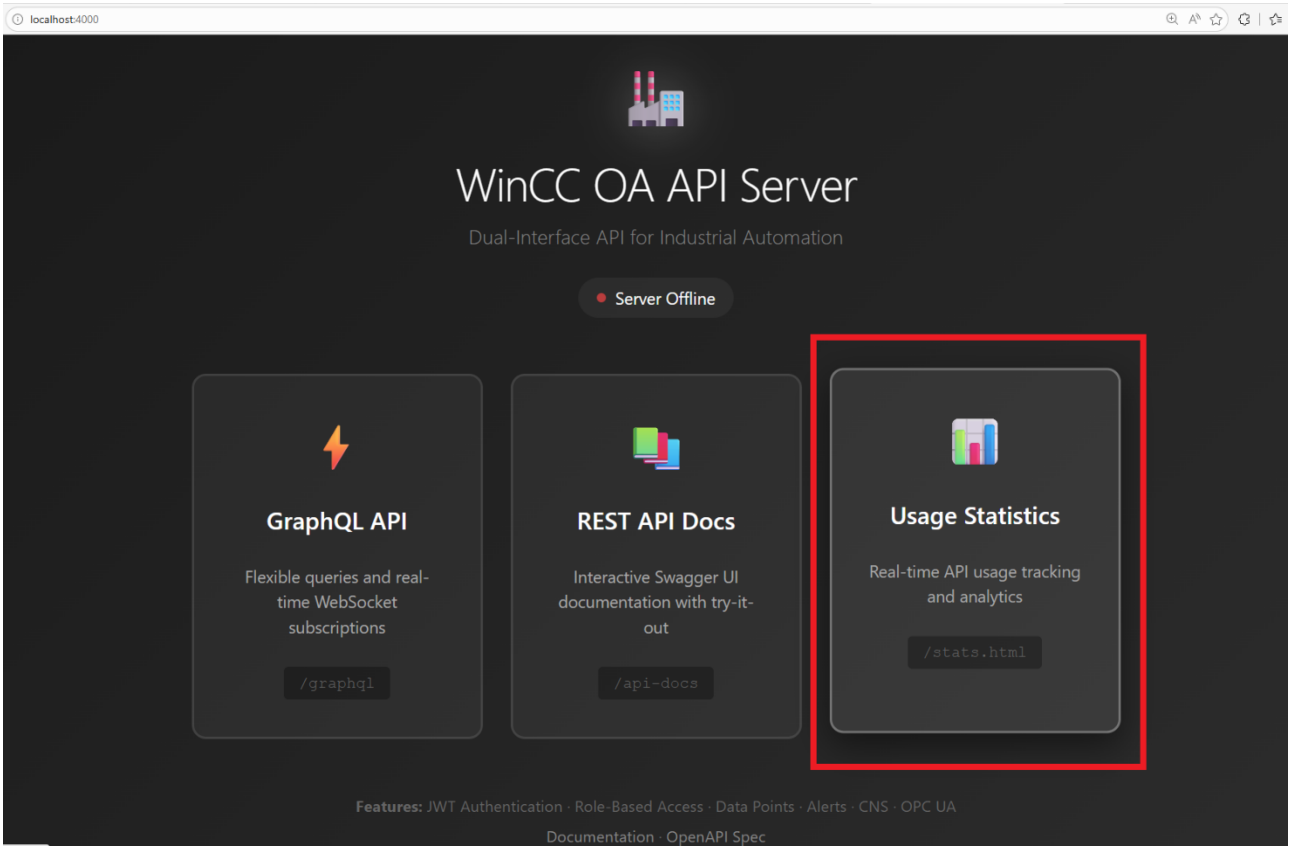


Figure 4-33 Accessing the API usage statistics page from the landing page

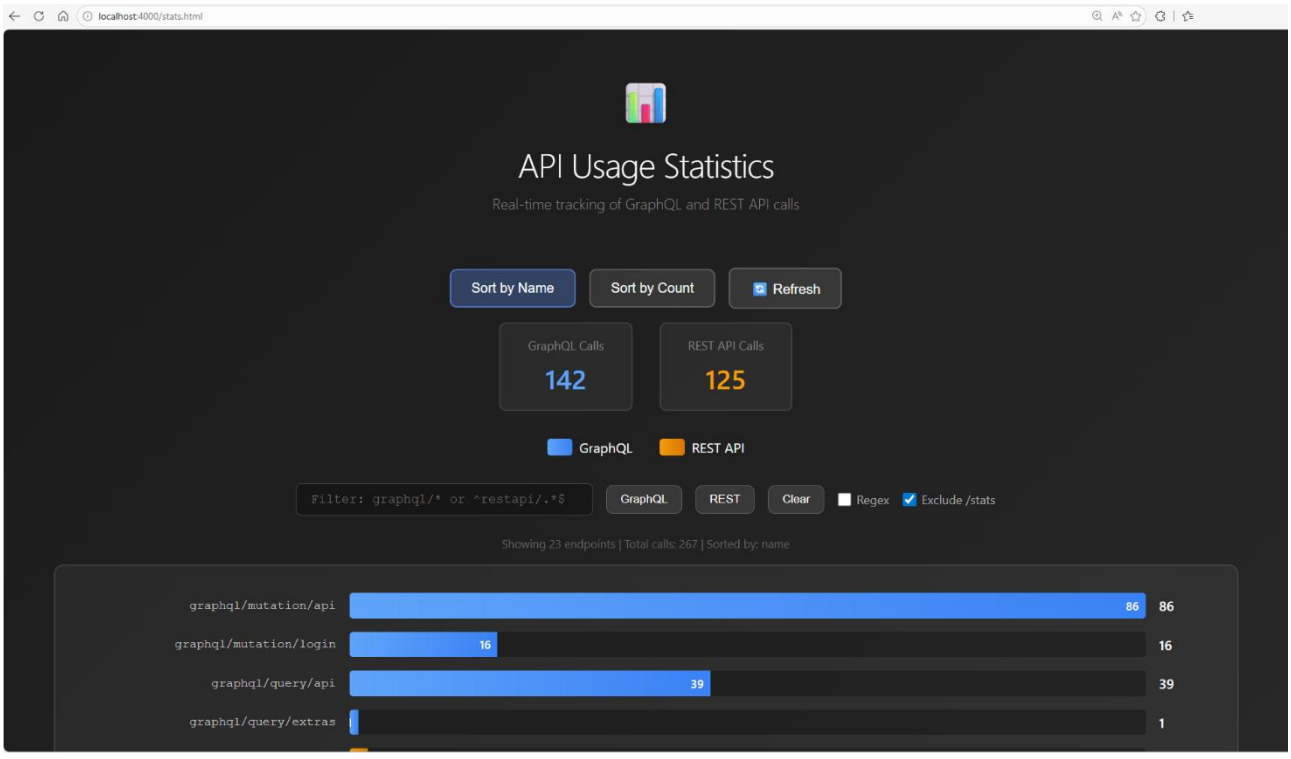


Figure 4-34 API usage statistics overview

The statistics view supports filtering and visualization of GraphQL and REST API endpoint activity.

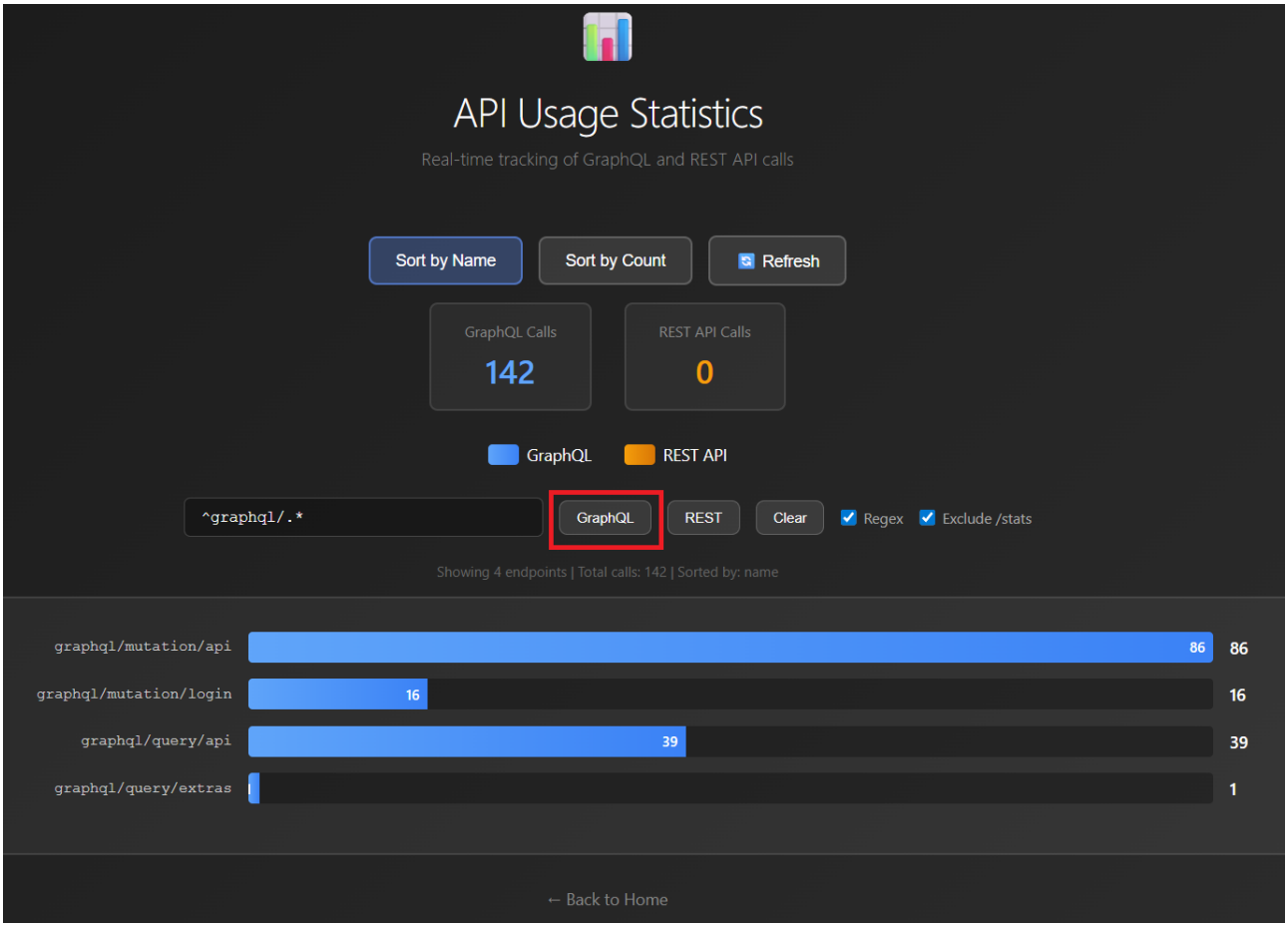


Figure 4-35 Filtering GraphQL and REST API statistics

Detailed endpoint statistics and request counters are displayed below the overview section.

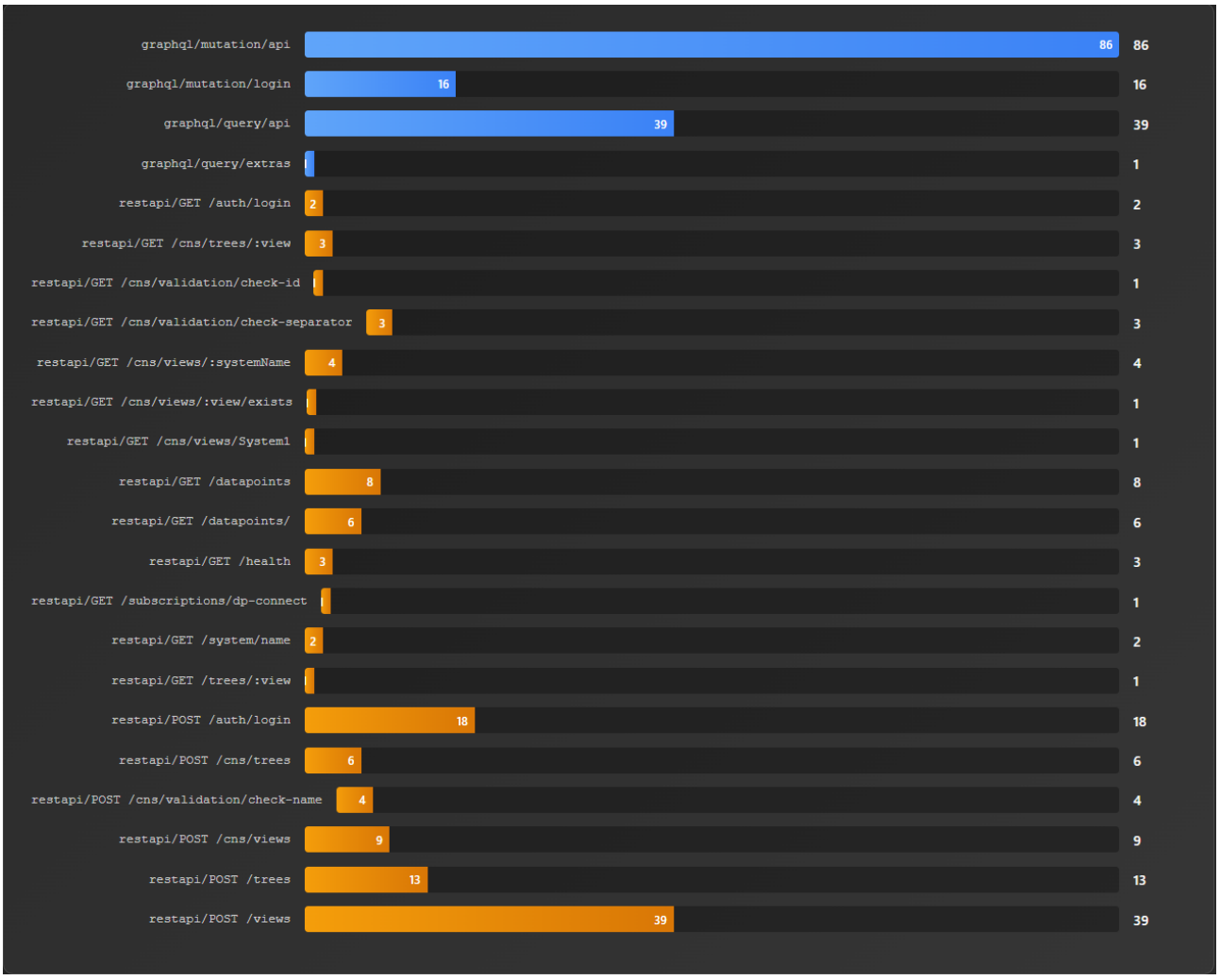


Figure 4-36 Detailed API endpoint statistics

5. Appendix

1.1 Important WinCC OA specific abbreviations

Acronym	Long form	Meaning
WinCC OA	Simatic WinCC Open Architecture	A SCADA system for visualizing and operating of processes, production flows, machines and plants in all lines of business. Distributed systems enable any number of stand-alone systems, from 2 to 2048, to be linked via a network. Each subsystem can be configured either as a single-user or multi-user system, redundant or not, in each case.
DPT	Data point type	Object definition (class) of a structured data object as mapping of a real device. Single data points (instances) are derived from the DPT. Therefore, the data point type is a form of template.
DP	Data point	Structured, device-oriented data object as representation of a real device within the control system. A data point contains one or more data point elements (process variables).
DPE	Data point element	Single process information within a device-oriented data point. Each DPE corresponds to a value/state. In addition to the value, there are DPE attributes like time stamp, quality information or origin.
GEDI	Graphic Editor GEDI	Graphic Editor. It is used both for drawing of process images ("panels") as well as for designing of symbols, dialogs, and scripting.
PARA	Configuration Tool	Editor for the creation and configuration of data point types, data points, and data point elements as well as their configs.
ASCII	American Standard Code for Information Interchange	Standardized protocol for storage and transfer of characters/text. In WinCC OA, the acronym also refers to the database import/export manager. It is a module to export and import configurations as ASCII files. Mass configuration can therefore be carried out in a spread sheet program (for example MS Excel), file editor, or in an external database.
D	Driver Manager ("Driver")	Interface for connecting controllers (PLC, DDC, ...) fieldbuses and telecontrol systems. A driver handles the communication via an external protocol and enables the exchange of information with WinCC OA. The processing of data from the “field” to WinCC OA contains event orientation, old/new comparison, transformation, conversion, and smoothing. The protocol of the Driver must be the same as the protocol of the “field” device. Furthermore, the connection (how to reach the device) must be configured in WinCC OA. For exchanging data, a periphery address must be configured on a corresponding DPE.
UNS	Unified Name Space	UNS (Unified Namespace) is a central, real-time data architecture that serves as a single source of truth for all data in an industrial or enterprise environment. It organizes and exposes data from various systems (e.g., PLCs, SCADA, MES, ERP) in a standardized, hierarchical structure, often using MQTT and Sparkplug B.

CNS	Common Name Service CNS	The plantmodel editor can be used for building and modifying data structures such as views and trees based on CNS and use them within projects easily.
CTRL	Control Manager ("Scripting")	Processing unit that allows to process user specific logic / business logic (control scripts). Control possesses an easy to learn syntax (similar to ANSI-C) and is processed by an interpreter (CTRL Manager).

Table 4-1 WinCC OA specific abbreviations

1.2 Service and support

1.2.1.1 WinCC OA Extended Services

Do you have questions about WinCC OA projects, need additional features, or require technical assistance? ETM provides 24/7 access to our complete service and support expertise for WinCC OA.

Our range of services includes the following:

- Extended Services provide tailored support for your evolving needs
- All kind of analysis/troubleshooting for older WinCC OA versions than our current mainline
- Project startup workshop
- Architecture definition
- Project engineering assistance with dedicated contact person
- Special project developments (special requirements, web widgets, gateways, etc.)
- WinCC OA library development assistance
- Project or architecture reviews with report
- Project-specific problem or performance analysis
- Project upgrade (analysis with report, assistance during upgrade, etc.)
- Assistance for complex error reproduction scenarios
- On-site assistance for any tasks related to WinCC OA and their components
- 24/7 on-duty assistance or priority callback for certain time range
- Database support Oracle®, InfluxDB®, PostgreSQL® and MS SQL®
- Raima/HDB to SQLite/NGA migration
- Setup and consulting for WinCC OA Add-ons e.g., APM, AMS, DRS, ...
- WinCC OA Security services (as per the WinCC OA Security Guideline, NIS2)
- Tests on unsupported platforms (e.g., unsupported OS)
- Individual Workshops (driver workshop, UI workshop, business logic, etc.)
- Factory Acceptance Test (FAT)/Site Acceptance Test (SAT) assistance
- Creating of prototypes, proof of concepts, demos, etc.
- Project tender analysis and evaluation of projects

  You can find detailed information on our range of services in the service catalog web page:

www.winccoa.com/documentation/WinCCOA/latest/en_US/Support/topics/support_extendedServices.html

1.2.1.2 SiePortal

The integrated platform for product selection, purchasing and support - and connection of Industry Mall and Online support. The SiePortal home page replaces the previous home pages of the Industry Mall and the Online Support Portal and combines them.

- Products & Services
  In Products & Services, you can find all our offerings as previously available in Mall Catalog.
- Support
  In Support, you can find all information helpful for resolving technical issues with our products.
- mySieportal
  mySiePortal collects all your personal data and processes, from your account to current orders, service requests and more. You can only see the full range of functions here after you have logged in.

You can access SiePortal via this address:

sieportal.siemens.com

1.2.1.3 Technical Support

The Technical Support of Siemens Industry provides you fast and competent support regarding all technical queries with numerous tailor-made offers.
– ranging from basic support to individual support contracts. Please send queries to Technical Support via Web form:

siemens.com/SupportRequest

1.2.1.4 WinCC OA – Training and Certification

To fully leverage the flexibility and openness of WinCC OA, we offer a wide range of training courses, from beginner to expert levels. Our modules cover various topics, with options for individual training. For WinCC OA partners, completing specific courses is required to obtain or maintain partner status, ensuring the highest level of expertise and support.

For more information on our offered trainings and courses, as well as their locations and dates, refer to our web page:

www.winccoa.com/support/training.html

1.3 Change documentation

Version	Date	Modifications
V1.0	4/2025	First version

Table 4-2 Change Documentation

Published by
Siemens AG
DI FA HMI ISW ETM

Marktstrasse 3
7000 Eisenstadt
Austria
E-mail: wincc_oa.at@siemens.com
Web: www.siemens.com/wincc-open-architecure

For the U.S. published by
Siemens Industry Inc

100 Technology Drive
Alpharetta, GA 30005
United States